

Final Report: End-to-End Telemetry Anomaly Detection with Laptop-to-Raspberry Pi Streaming

1. Project Overview

This project implements an end-to-end anomaly detection system for telemetry time series. Models are trained and evaluated offline on a laptop, then deployed as saved artifacts to a Raspberry Pi for streaming inference. The evaluated model set is `'zscore'`, `'iforest'`, `'lstm_ae'`, and `'cnn_ae'`, and the datasets are SKAB and NAB `'realAWSCloudwatch'`.

Executive Summary

This project combines offline model development on a laptop with streaming inference on a Raspberry Pi. The offline stage includes data loading, preprocessing, feature engineering, training, threshold calibration, evaluation, and reporting, while the online stage uses FastAPI and Tailscale so the Pi can pull telemetry from the laptop and score it locally. Four models were compared: Z-score, Isolation Forest, LSTM-AE, and CNN-AE. The evaluation uses point-level metrics, event-level metrics, and PR-AUC so that both statistical accuracy and operational alert coherence are captured. Across the experiments, calibrated simple models performed very well on SKAB, especially when the stream contained a stable normal warmup segment. On the harder NAB `'realAWSCloudwatch'` series, the temporal autoencoders were generally stronger, particularly when the anomaly was embedded in noisier and less stationary cloud telemetry.

2. Offline Pipeline

The offline stage covers dataset loading, preprocessing, feature engineering, training, threshold calibration, evaluation, and visualization. The pipeline was designed so that the same feature assumptions used offline can be reproduced later during streaming inference on the Raspberry Pi.

2.1 Data Characteristics

The two datasets stress the system in different ways.

- SKAB is closer to a controlled industrial setting. Training uses the anomaly-free file and testing uses `'valve1/1.csv'`, so the learning problem is clean: learn normal behaviour first, then detect a fault in a separate operational trace.
- NAB `'realAWSCloudwatch'` is operationally noisier. The AWS series are less stationary, less uniform, and more ambiguous at the event boundary level, which is why event-level metrics matter strongly.

In practical terms, SKAB is the clearest dataset for threshold calibration and stable alerting, while NAB is the better test for robustness to drift, trend, and cloud-style telemetry noise.

2.2 Preprocessing and Feature Engineering

The preprocessing path includes timestamp parsing, missing-value handling, interpolation, rolling detrending, smoothing and standardization where configured, lag embeddings, rolling statistics, FFT features, and optional STL seasonality review.

The base configuration uses a sliding window of `60` points with stride `1`, standardization, lag steps `[1, 5, 10]`, and FFT energy features. These choices allow the baseline models to use short temporal context rather than only the current point.

Dataset-specific configuration matters:

- SKAB uses warmup-based threshold calibration during streaming because the normal prefix of the test stream provides a practical calibration region.
- NAB uses an anomaly-aware split, `drop\_labeled\_anomalies\_from\_train: true`, and a rolling detrend window of `144` samples so that slow cloud-metric shifts are less likely to dominate the anomaly score.

These decisions materially affect performance. In this project, preprocessing is not a minor implementation detail; it is part of the anomaly detector.

2.3 Model Roles

The four implemented models serve different roles rather than acting as interchangeable alternatives.

- Z-score is the simplest and most interpretable detector. It works well when anomalies create a clear deviation relative to recent normal behaviour, but it is sensitive to threshold calibration.
- Isolation Forest is a stronger baseline than thresholding because it combines engineered features nonlinearly, but it often fragments alerts when the anomaly score oscillates around the threshold.
- LSTM Autoencoder models longer temporal structure and is most useful when anomalous behaviour is expressed as a sequence pattern rather than a single extreme point.
- CNN Autoencoder also uses reconstruction error, but focuses more on local temporal motifs and short-range shape changes.

3. Raspberry Pi Deployment and Communication

Training remains on the laptop. The Raspberry Pi is used only for inference, which is the correct split for an edge-deployment workflow.

3.1 System Roles

Laptop / PC

- stores the datasets
- performs offline training and offline evaluation
- produces the artifact folders under `artifacts/<dataset>/<variant>`
- hosts the FastAPI telemetry service

Raspberry Pi

- clones the repository
- installs runtime dependencies through `pi\_setup.sh`
- receives the trained artifacts copied from the laptop
- pulls telemetry rows from the laptop API over Tailscale
- performs local inference with `infer\_stream\_pi.py`
- writes alert logs under `logs/`

This split matters because the Pi is the actual inference node. The laptop is the training and telemetry-source side of the system.

3.2 Raspberry Pi Setup

The validated Pi path is intentionally simple:

1. clone the repository on the Pi
2. run ``bash scripts/pi_setup.sh``
3. activate the virtual environment
4. run ``python scripts/pi_preflight.py --api-base-url http://<tailscale-host-or-ip>:8000``

``pi_setup.sh`` creates the virtual environment, upgrades ``pip``, installs the required packages, and prepares the output directories. ``pi_preflight.py`` checks imports, required files, Tailscale availability, and API reachability before inference starts.

3.3 Artifact Transfer

After offline training on the laptop, the Pi receives the trained artifacts rather than retraining models locally. The key files are:

- ``thresholds.json``
- ``metadata.json``
- ``iforest.pkl``
- ``zscore_params.pkl``
- ``lstm_ae.pt``
- ``cnn_ae.pt``
- ``seq_scaler.pkl``
- ``scaler.pkl`` where applicable for the baseline path

These files are copied with ``scp``, ``rsync``, or an equivalent transfer method.

3.4 Tailscale and FastAPI Communication

The laptop and Pi communicate through Tailscale, which creates a private tailnet and avoids exposing the telemetry API publicly. Once both devices join the same tailnet, the Pi reaches the laptop through its Tailscale hostname or IP, for example ``http://<laptop-tailnet-name>:8000``.

The laptop hosts a FastAPI service with two main endpoints:

- ``GET /health``
- ``GET /stream/next?cursor=0&batch_size=1``

``/health`` is used for service checks and ``/stream/next`` returns the next telemetry batch. In deployment terms, the laptop acts as the stream producer and the Pi acts as the stream consumer plus inference engine.

3.5 Streaming Inference on the Pi

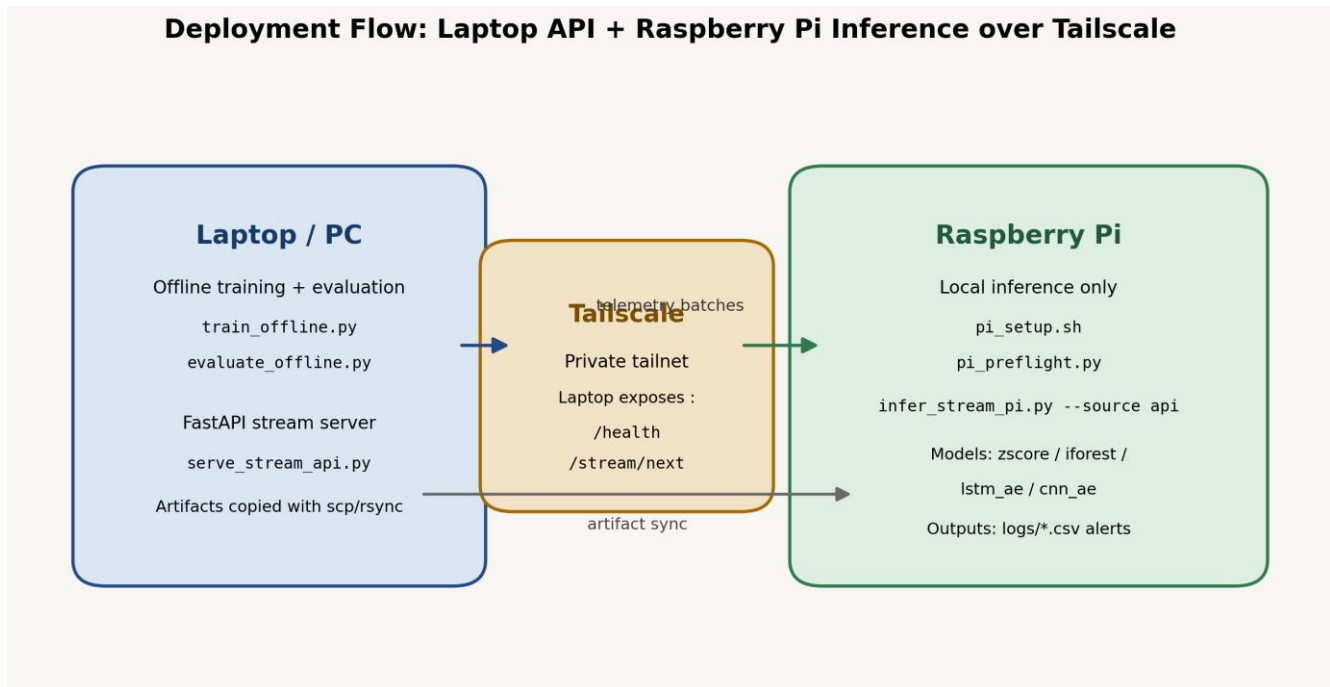
A representative Pi command is:

```
python scripts/infer_stream_pi.py --dataset skab --split anomalyfree_vs_valve1_1 --model lstm_ae --
source api --api-base-url http://<laptop-tailnet-name>:8000 --api-batch-size 32 --log-file
logs/pi_lstm_api.csv
```

When this runs, the Pi requests the next telemetry batch, updates its local sliding window, rebuilds the required features or sequences, applies the saved model and threshold locally, and writes alerts to CSV. To keep online inference consistent with offline training, the streaming path rebuilds baseline feature vectors using the feature-column schema stored in artifact metadata. This feature-schema consistency is essential for the saved scaler and model to remain valid.

3.6 Deployment Validation

The deployment was validated in stages: offline train/evaluate on the laptop, start the laptop API, verify Tailscale and API reachability from the Pi, copy artifacts to the Pi, run `infer\_stream\_pi.py` in API mode, and confirm that the expected alert CSV logs are produced. This staged path demonstrates a real split between offline development and edge execution.



Deployment architecture for laptop-to-Pi inference over FastAPI and Tailscale.

4. Current Results

4.1 Point-level F1

Variant	Z-score	Isolation Forest	LSTM-AE	CNN-AE
SKAB valve1/1	0.677	0.628	0.667	0.642
NAB rds_cpu	0.446	0.437	0.571	0.571
NAB ec2_cpu	0.242	0.223	0.302	0.302
NAB ec2_network	0.157	0.398	0.613	0.173

4.2 Event-level F1

Variant	Z-score	Isolation Forest	LSTM-AE	CNN-AE
SKAB valve1/1	1.000	0.681	0.571	0.667
NAB rds_cpu	1.000	1.000	1.000	1.000
NAB ec2_cpu	1.000	0.237	1.000	1.000
NAB ec2_network	0.400	0.047	1.000	1.000

4.3 Best model by metric

Variant	Best point-F1 model	F1	Best PR-AUC model	PR-AUC	Best event-F1 model	Event F1
SKAB valve1/1	Z-score	0.677	LSTM-AE	0.695	Z-score	1.000
NAB rds_cpu	LSTM-AE	0.571	LSTM-AE	0.493	All models	1.000
NAB ec2_cpu	LSTM-AE	0.302	Z-score	0.759	Z-score / LSTM-AE / CNN-AE	1.000
NAB ec2_network	LSTM-AE	0.613	LSTM-AE	0.950	LSTM-AE / CNN-AE	1.000

5. Results Interpretation

5.1 SKAB

SKAB is the clearest evaluation case because it separates normal training data from a fault-containing test file. The best point-level F1 is achieved by calibrated Z-score (0.677), while LSTM-AE is close (0.667). LSTM-AE has the strongest ranking quality with PR-AUC 0.695 and ROC-AUC 0.743, but Z-score remains the cleanest alerting model because it turns the anomaly into one coherent event.

The detailed numbers explain that difference. Z-score produces one predicted event and reaches event-level F1 `1.000`, whereas Isolation Forest generates `91` predicted events and the two autoencoders split the fault into several segments. SKAB therefore shows that the best ranking model and the best operational alerting model are not always the same.

5.2 NAB `rds\_cpu\_utilization\_cc0c53`

This is the clearest NAB success case for the advanced models. LSTM-AE and CNN-AE both reach point-level F1 0.571, clearly ahead of the baseline models. All four methods reach event-level F1 `1.000`, so the main difference is not whether the event is detected, but how tightly it is localized point by point.

This pattern is consistent with the intended role of the autoencoders. Once the anomaly has a temporal signature rather than just a single extreme excursion, the sequence models describe the abnormal region more precisely than the baseline detectors.

5.3 NAB `ec2\_cpu\_utilization\_5f5533`

This series is harder. The autoencoders again perform best with F1 0.302, compared with 0.242 for Z-score and 0.223 for Isolation Forest. Event-level F1 remains `1.000` for Z-score and both autoencoders, which means the anomalous period is detected coherently even though the pointwise boundary remains broad.

This split also shows why F1 should not be read alone. Z-score has the highest PR-AUC on this series, but its final threshold is too permissive and creates an overly wide anomaly region. Isolation Forest performs worst not because it misses the anomaly completely, but because it fragments the alert stream heavily.

5.4 NAB `ec2\_network\_in\_257a54`

The refreshed network run adds important evidence rather than acting as a placeholder. The strongest result is LSTM-AE, which reaches point-level F1 0.613, PR-AUC 0.950, and event-level F1 1.000. This is the cleanest outcome on the split because the model predicts one coherent anomaly event with perfect precision.

Isolation Forest reaches the second-best point-level F1 (0.398), but its event-level F1 collapses to 0.047 because it fragments the anomaly into many predicted events. Z-score and CNN-AE both detect the event at the event level, but they are less precise pointwise. This reinforces the same conclusion seen elsewhere in NAB: once the anomaly is embedded in noisy cloud telemetry, the sequence model becomes the most reliable detector.

5.5 Overall Interpretation

Across the labeled anomaly cases, the advanced models add the most value on the NAB series. On SKAB, a well-calibrated simple model remains extremely competitive. Another important pattern is that the models differ not only in statistical performance, but also in alert behaviour: Z-score tends to produce broad but coherent anomaly regions, Isolation Forest often fragments alerts, and the autoencoders usually perform best when the anomaly is truly temporal.

For deployment, this means the preferred model is not simply the one with the highest point-level F1. The better choice depends on whether the system prioritizes stable event-level alerting, fine-grained localization, ranking quality, or a balance across all three.

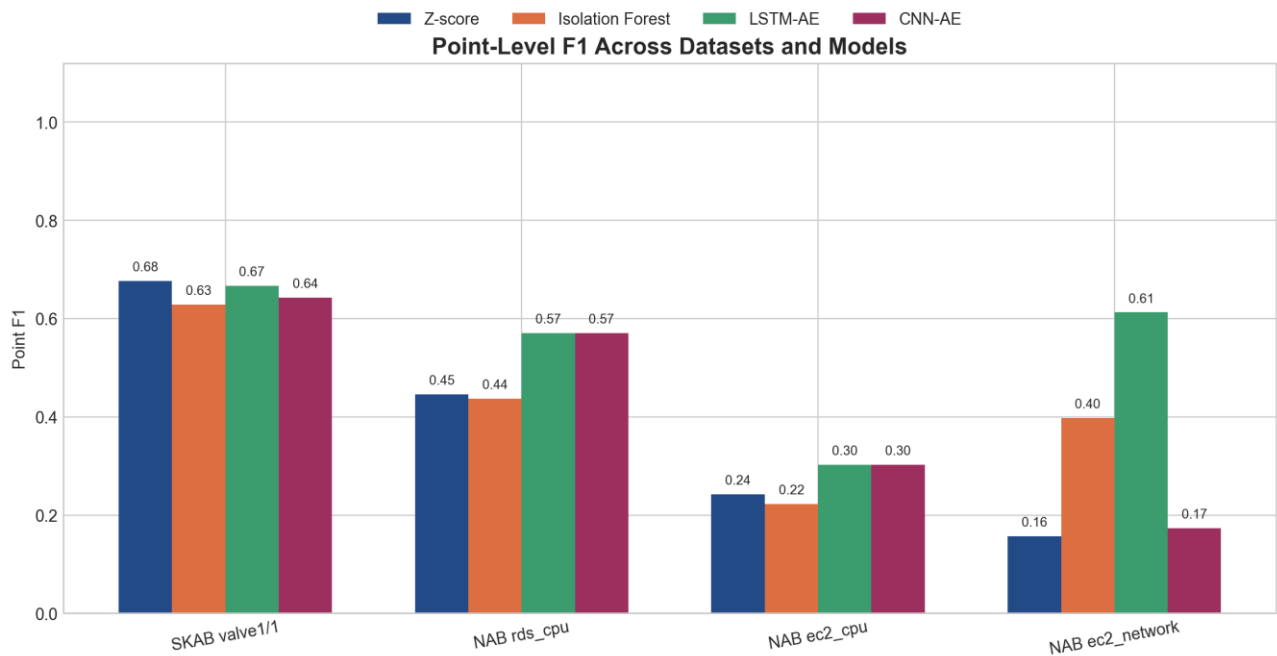
6. Figure Appendix and Visual Discussion

This section retains the full figure set while avoiding repeated plot-by-plot prose. The summary tables in Section 5 remain the main source for exact metric values; the figures below are used to interpret anomaly shape, alert fragmentation, and the effect of thresholding.

All confusion matrices should be read in the same way. High recall with lower precision usually means the model covers the anomaly broadly but raises many false positives. Low recall indicates missed anomalous points. A large number of predicted events indicates fragmentation into many short alerts, while event-level F1 measures whether those point predictions still correspond to one coherent operational incident.

6.1 Overall Comparison Figures

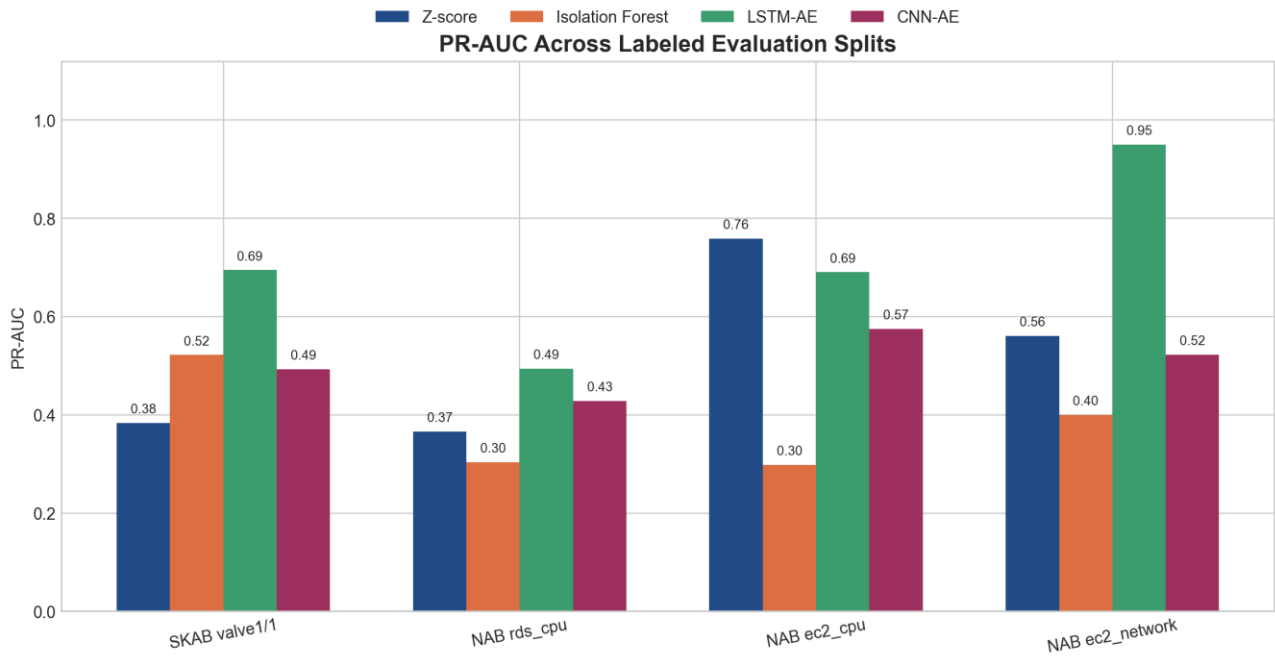
The summary figures show the high-level pattern of the project without the noise of per-run detail. Point-level F1 identifies the strongest pointwise detector on each split, event-level F1 highlights alert coherence, and PR-AUC separates ranking quality from the final deployed threshold. Together they show a clear division: SKAB favours a calibrated simple detector, while the harder NAB cloud telemetry is generally handled better by the temporal autoencoders. The refreshed `ec2_network` run strengthens that conclusion because LSTM-AE is clearly strongest there, whereas Isolation Forest remains visibly fragmented.



Point-level F1 comparison across datasets and models.



Event-level F1 comparison across datasets and models.

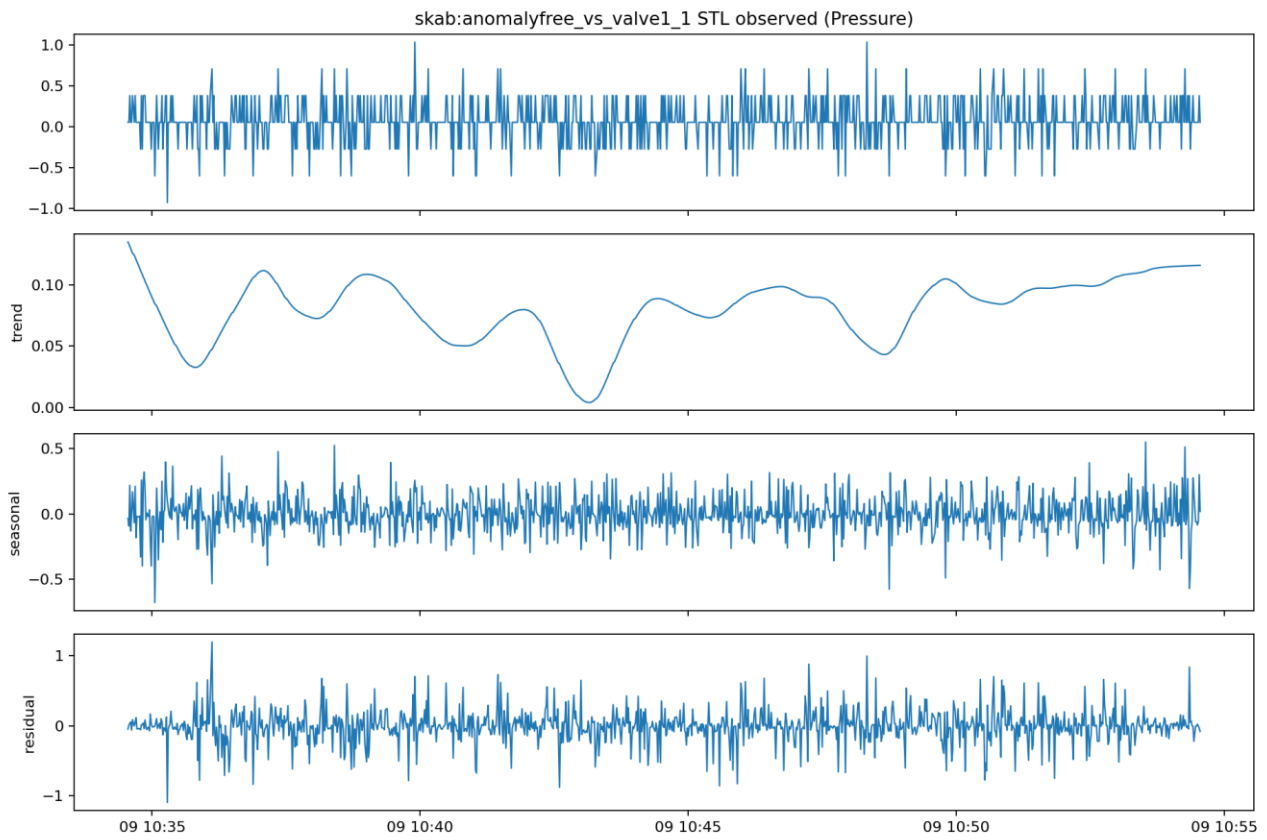


PR-AUC comparison across labeled evaluation splits.

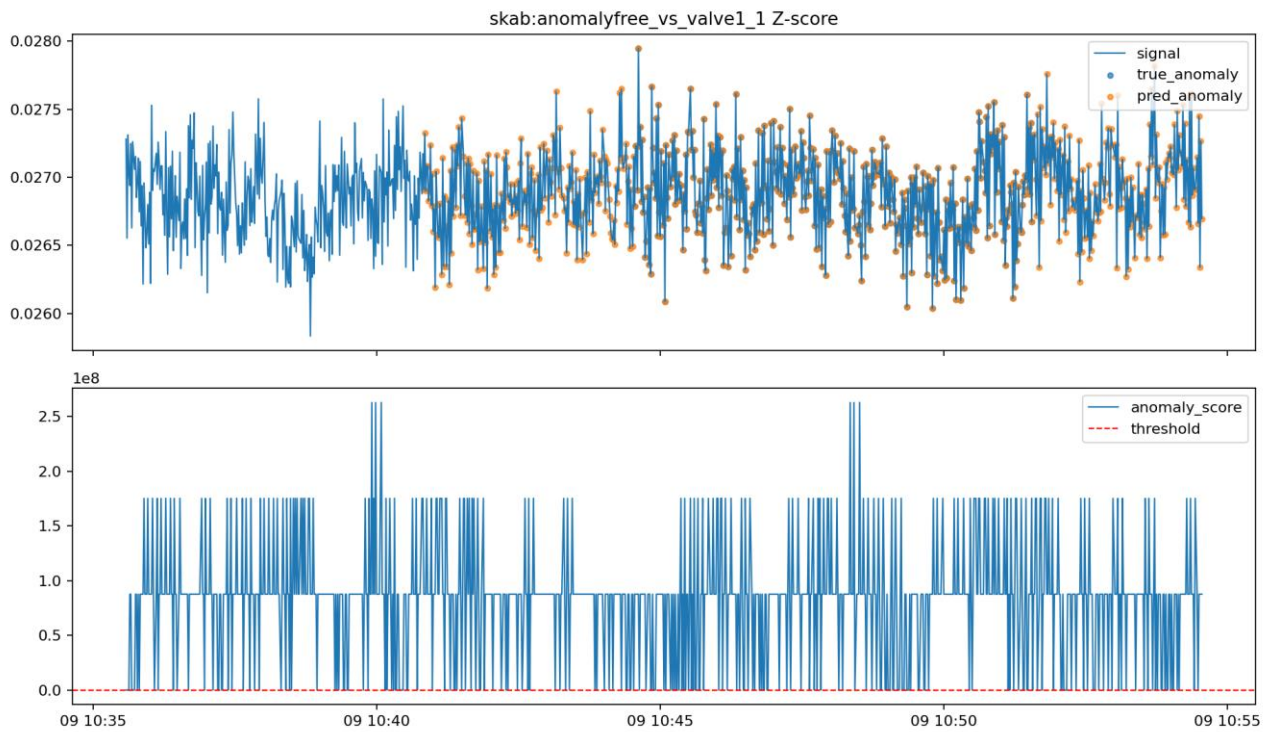
6.2 SKAB Figures

The SKAB figures confirm why this dataset is the clearest operational benchmark in the project. The STL decomposition shows a stable baseline before the fault interval, which supports both warmup-based calibration and sequence modeling. Visually, Z-score is the cleanest alerting model because it produces one coherent event with full recall. Isolation Forest remains competitive at the point level but fragments heavily. The autoencoders separate the anomaly well, yet they split the event into multiple segments, which is why their event-level performance trails their ranking quality.

This contrast is central to the report: the best ranking model is not always the best alerting model. In SKAB, calibrated thresholding remains the most operationally stable option even though LSTM-AE has the strongest PR-AUC.

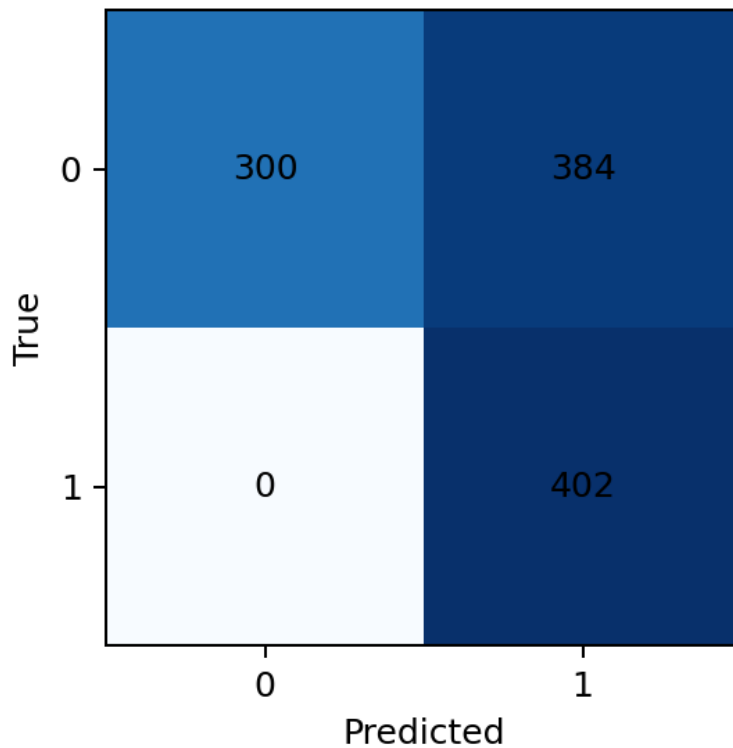


SKAB STL decomposition: stable baseline before the fault interval.

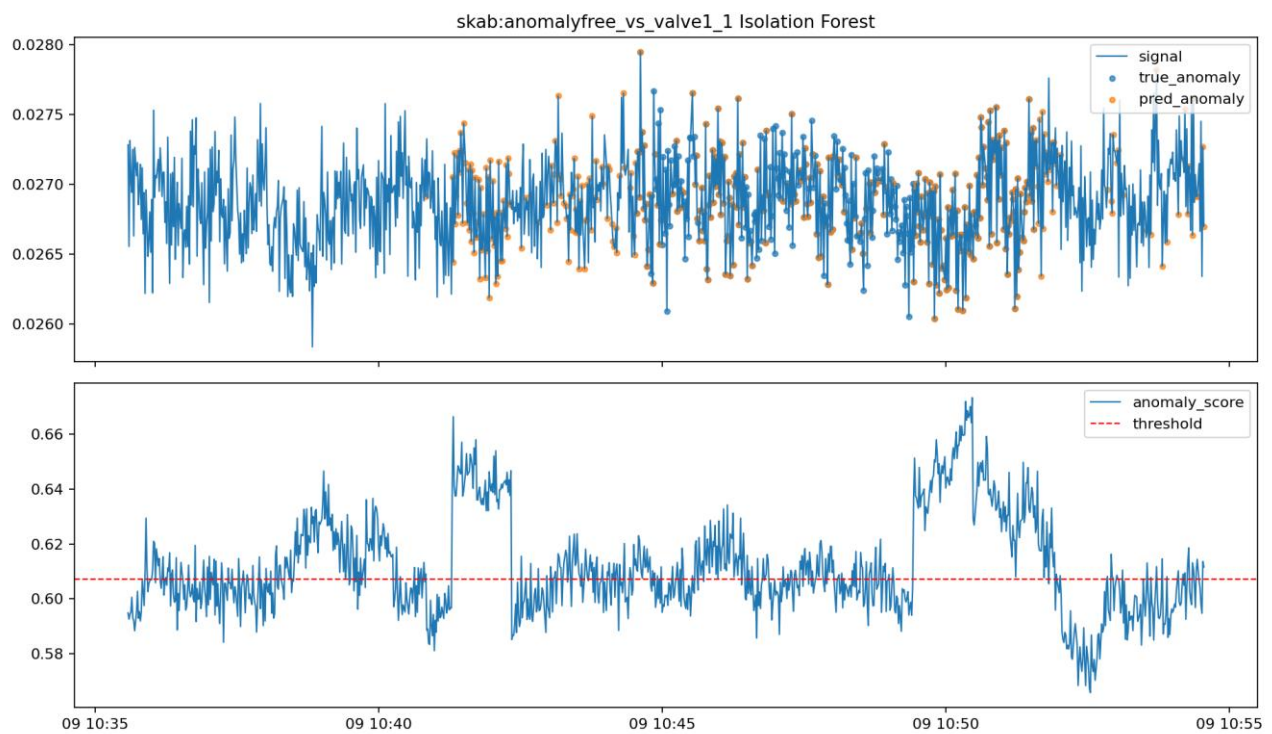


SKAB Z-score: coherent event detection with high recall.

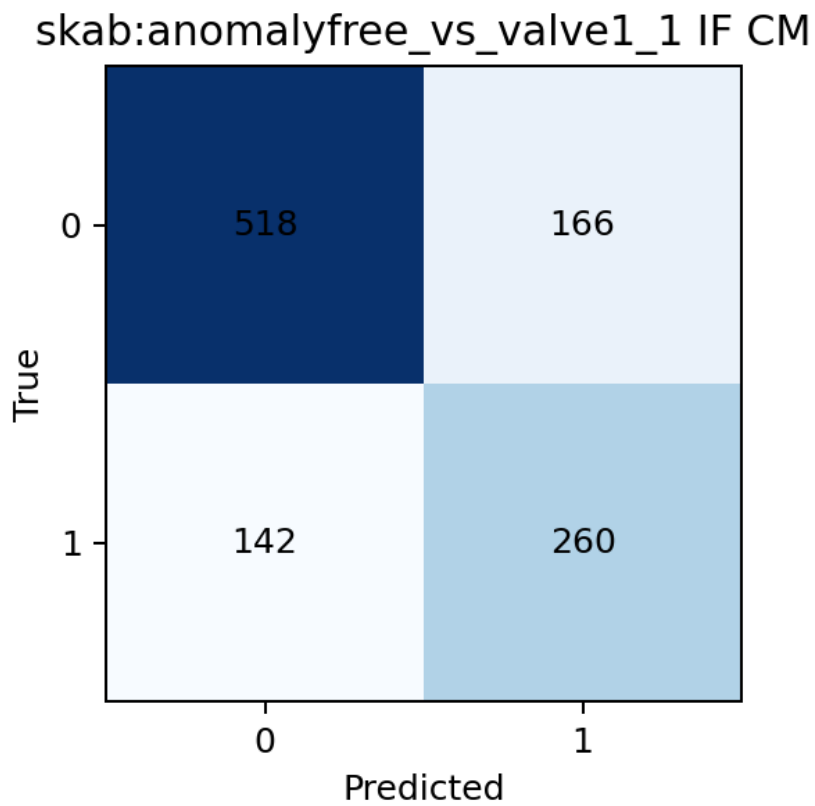
skab:anomalyfree_vs_valve1_1 Z-score CM



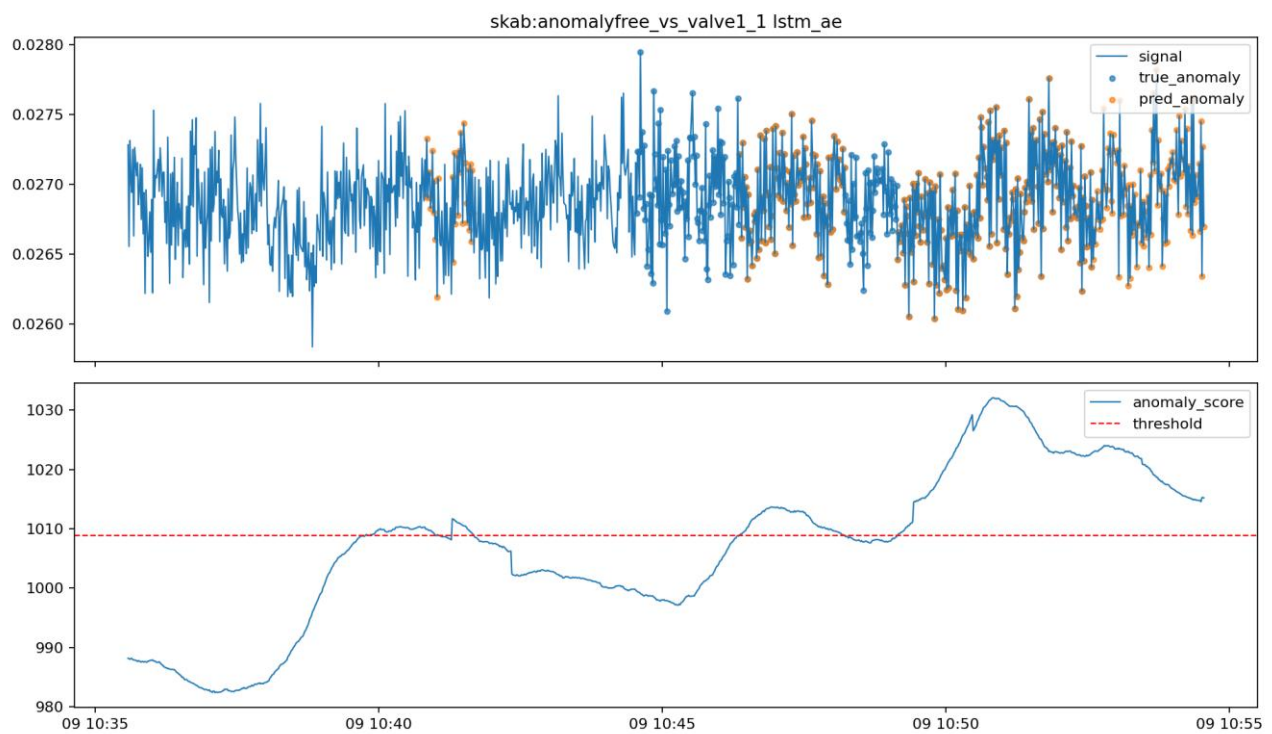
SKAB Z-score confusion matrix: full recall with lower precision.



SKAB Isolation Forest: competitive point-level detection but fragmented alerts.

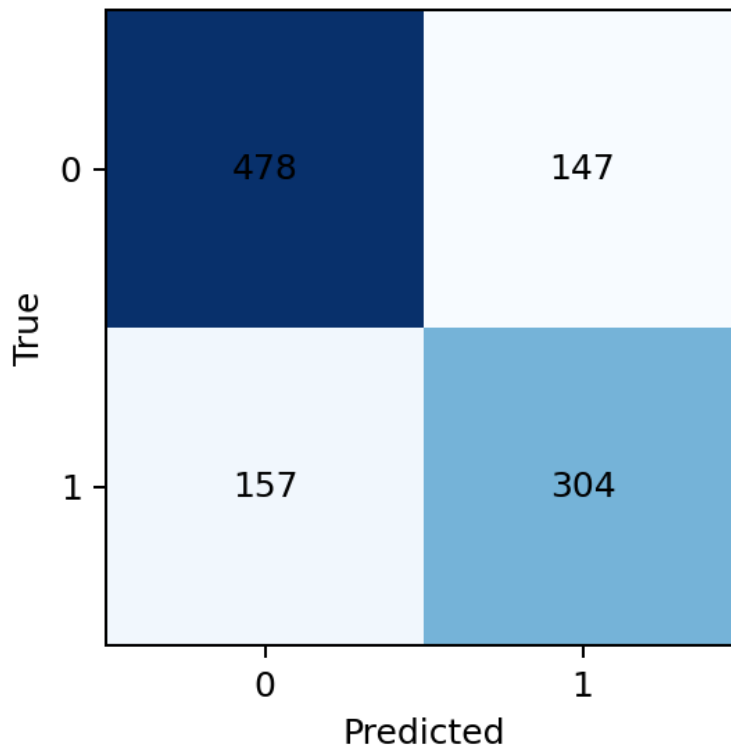


SKAB Isolation Forest confusion matrix: balanced recall, lower event coherence.

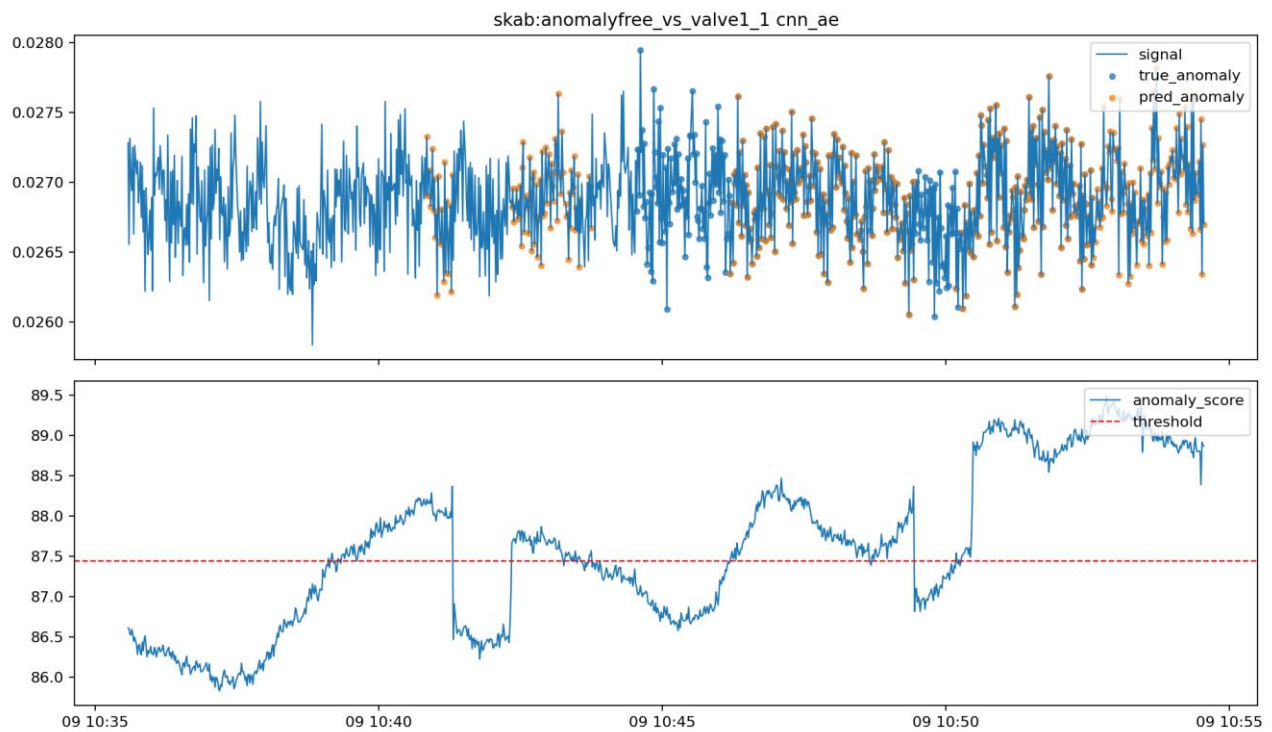


SKAB LSTM-AE: strong ranking quality with moderate fragmentation.

skab:anomalyfree_vs_valve1_1 lstm_ae CM

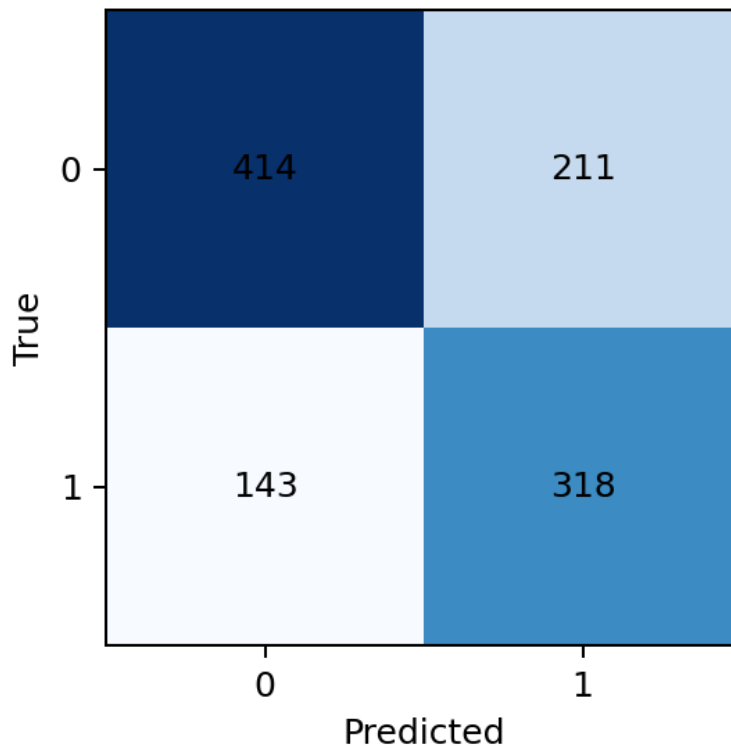


SKAB LSTM-AE confusion matrix: balanced precision and recall.



SKAB CNN-AE: local temporal detection with several short alerts.

skab:anomalyfree_vs_valve1_1 cnn_ae CM

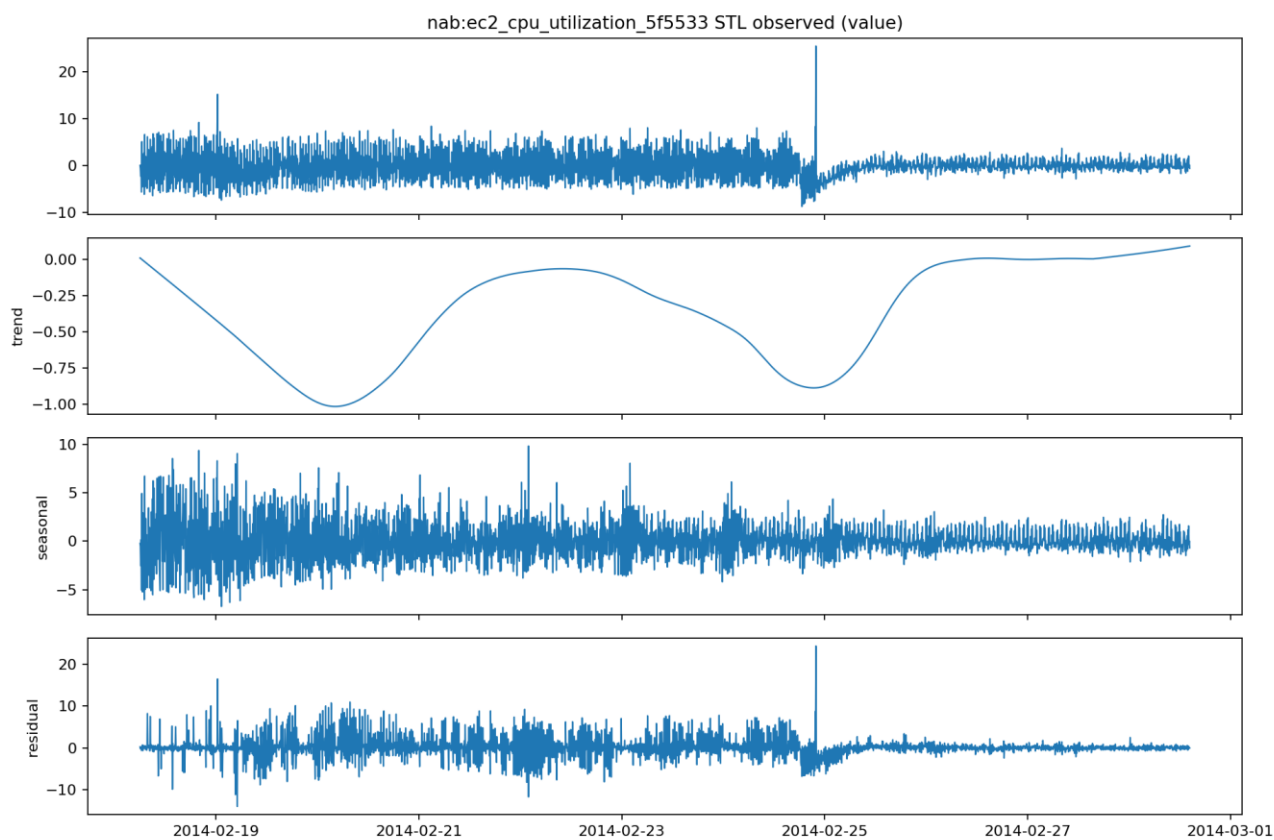


SKAB CNN-AE confusion matrix: good recall with moderate false positives.

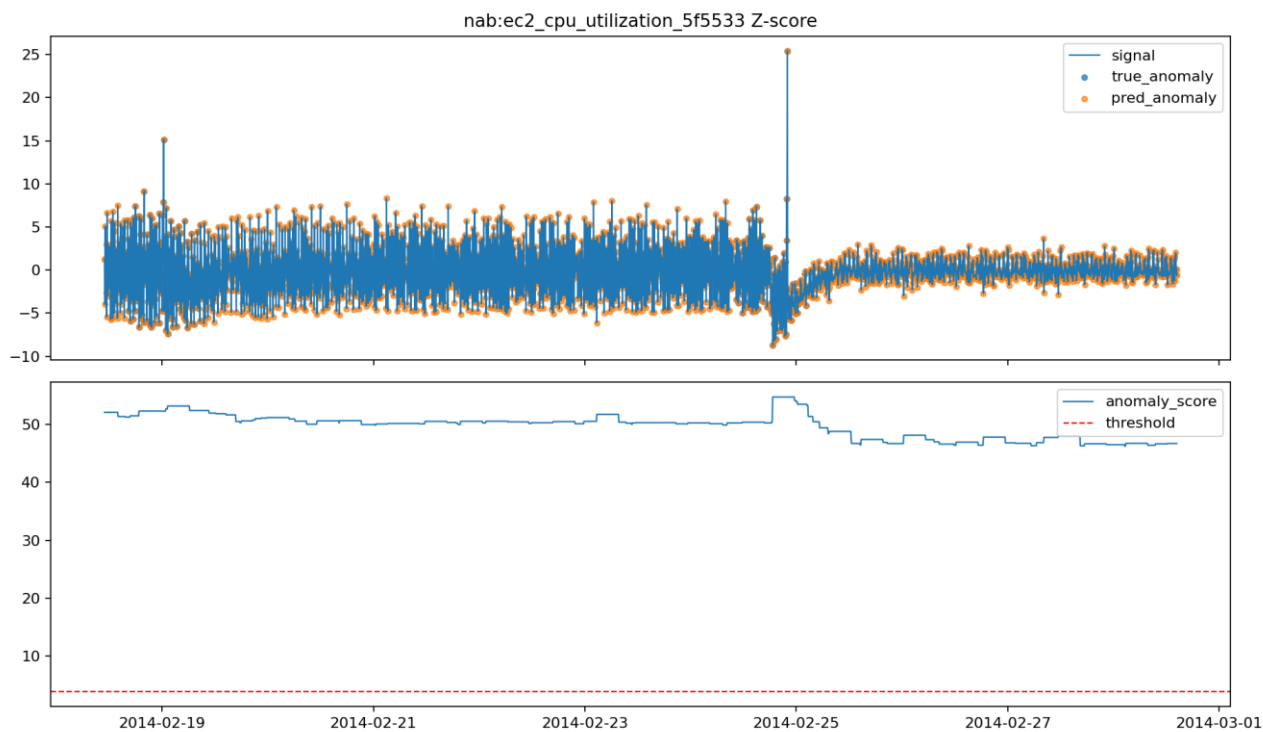
6.3 NAB EC2 CPU Figures

The EC2 CPU series is harder because the anomaly sits inside structured cloud telemetry rather than a clean normal-versus-fault transition. The seasonality plot shows periodic behaviour together with broader level shifts, so the detector must separate recurring variation from a sustained abnormal region. In the score plots, Z-score and both autoencoders produce one broad event, while Isolation Forest becomes highly fragmented and loses event coherence.

The figures also explain why the ranking and thresholded views differ. Z-score ranks this split extremely well, but its deployed threshold produces an overly wide anomaly region. The autoencoders preserve one coherent event while improving the point-level boundary, which is why they are the strongest models on this series.

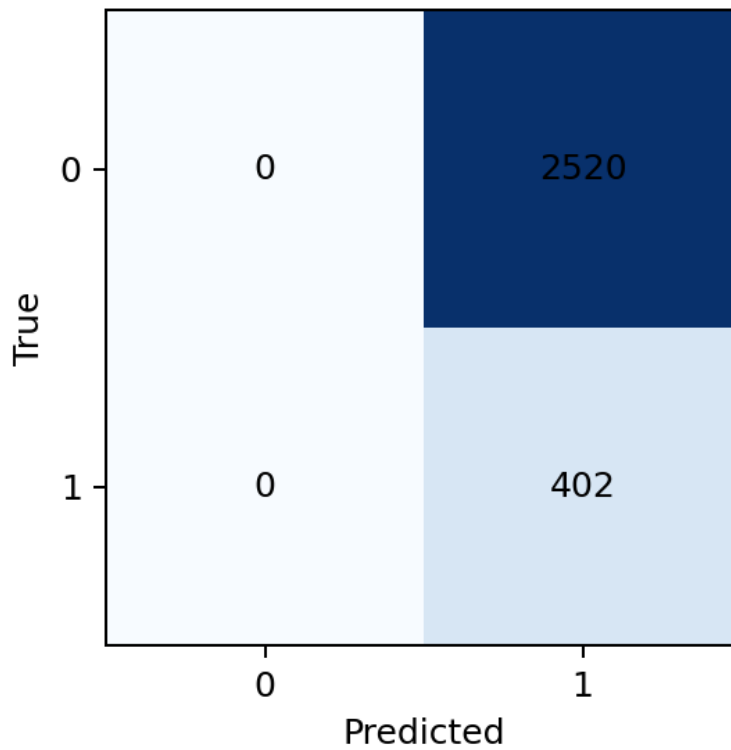


NAB EC2 CPU STL decomposition: periodic structure and level shifts.

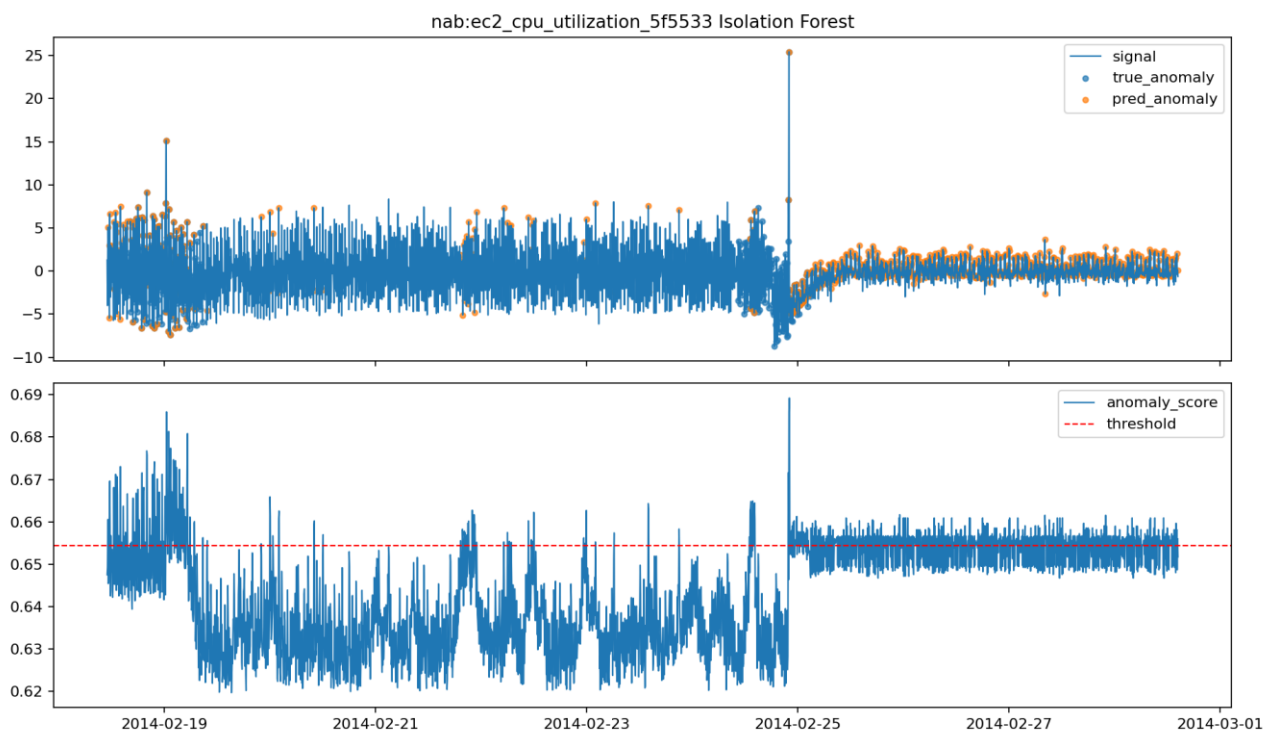


NAB EC2 CPU Z-score: broad but coherent anomaly region.

nab:ec2_cpu_utilization_5f5533 Z-score CM

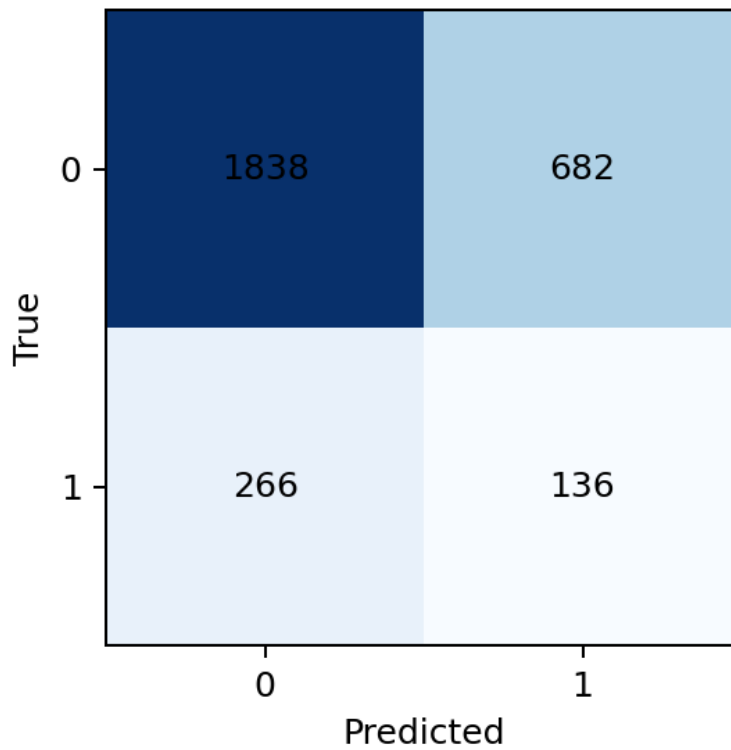


NAB EC2 CPU Z-score confusion matrix: full recall with many false positives.

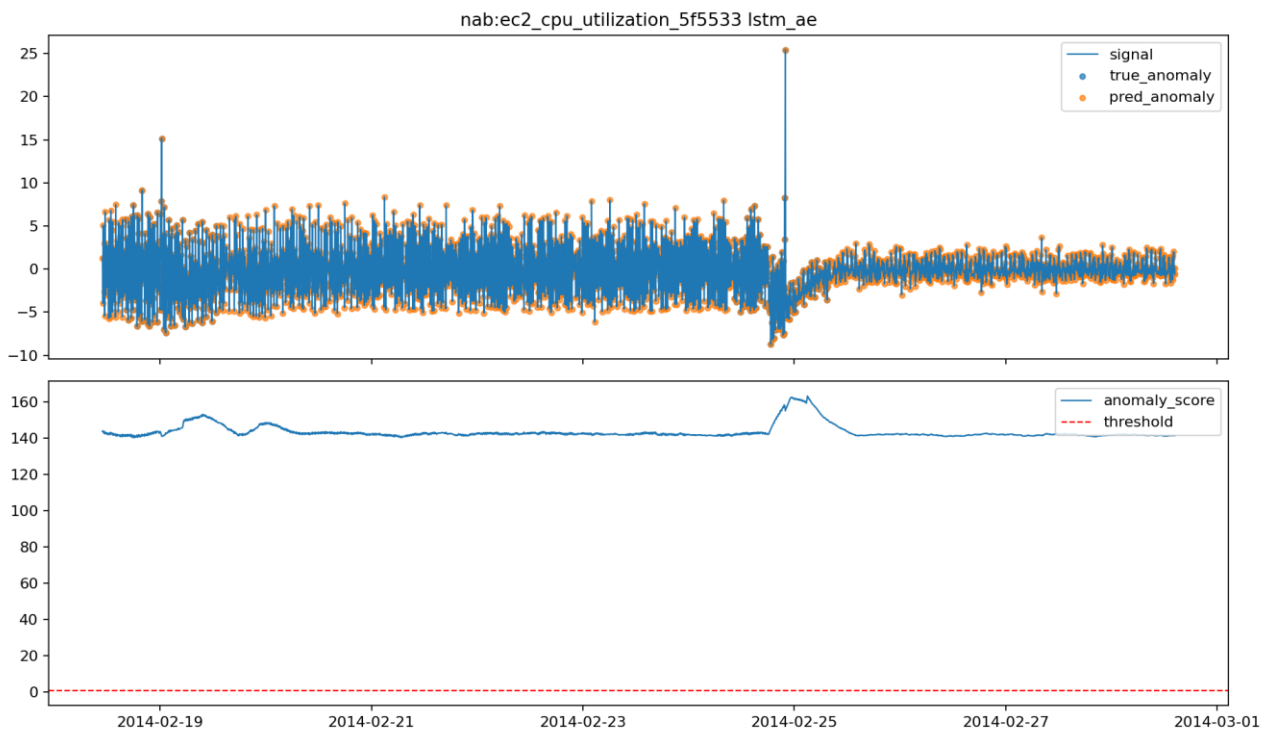


NAB EC2 CPU Isolation Forest: fragmented alerting across the anomaly window.

nab:ec2_cpu_utilization_5f5533 IF CM

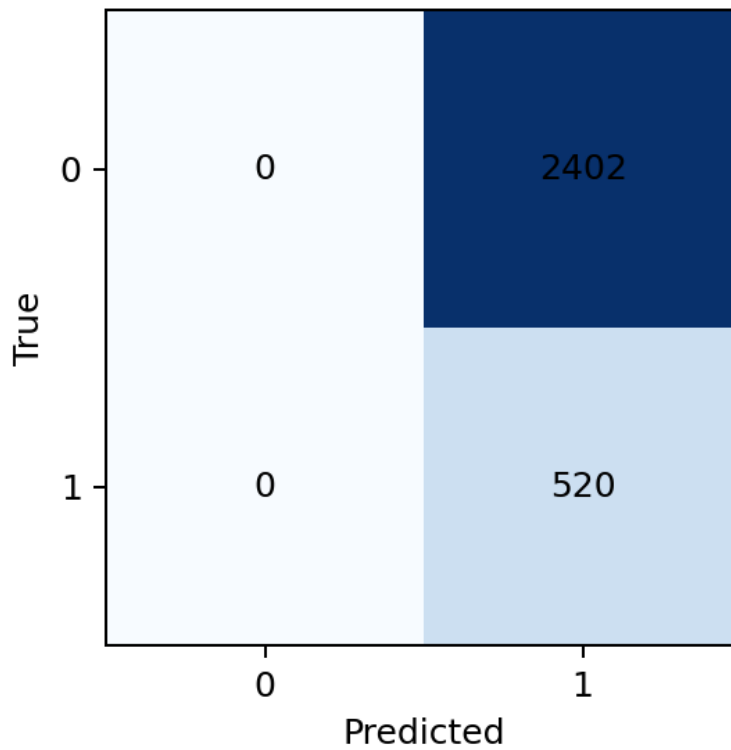


NAB EC2 CPU Isolation Forest confusion matrix: reduced recall and weak event coherence.

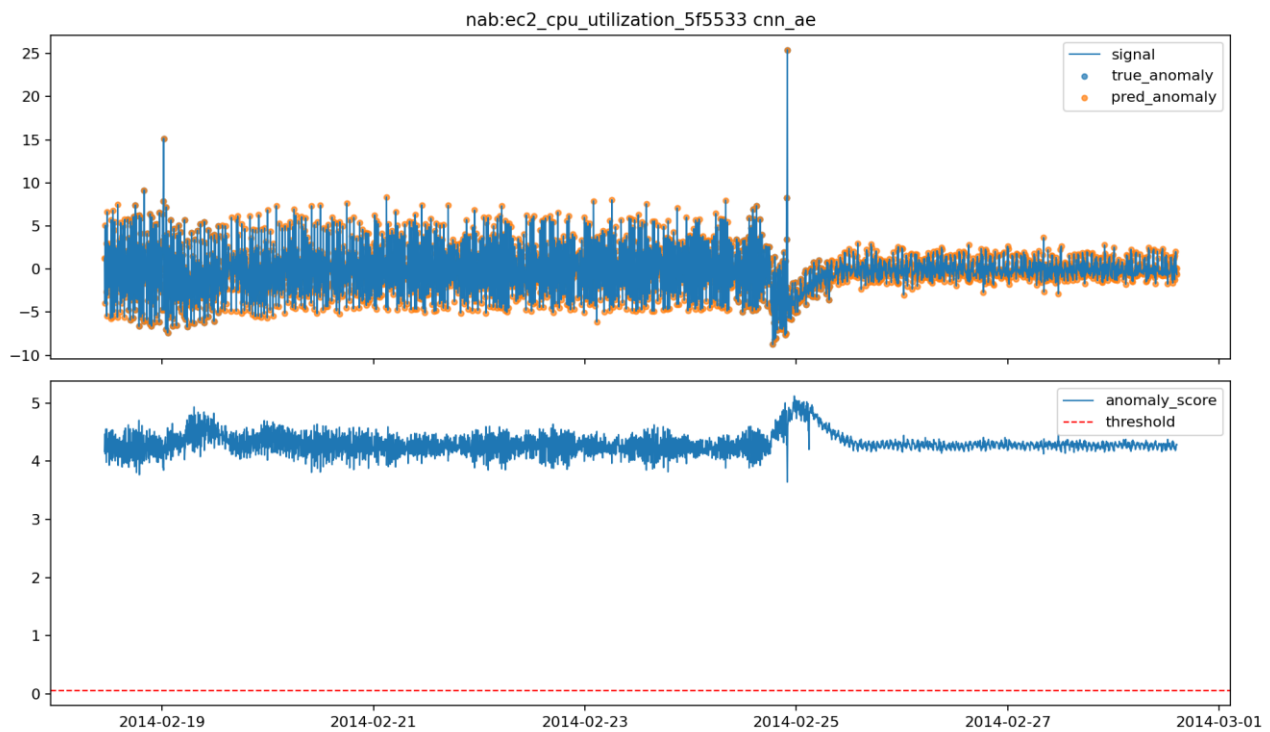


NAB EC2 CPU LSTM-AE: one coherent sequence-level anomaly.

nab:ec2_cpu_utilization_5f5533 lstm_ae CM

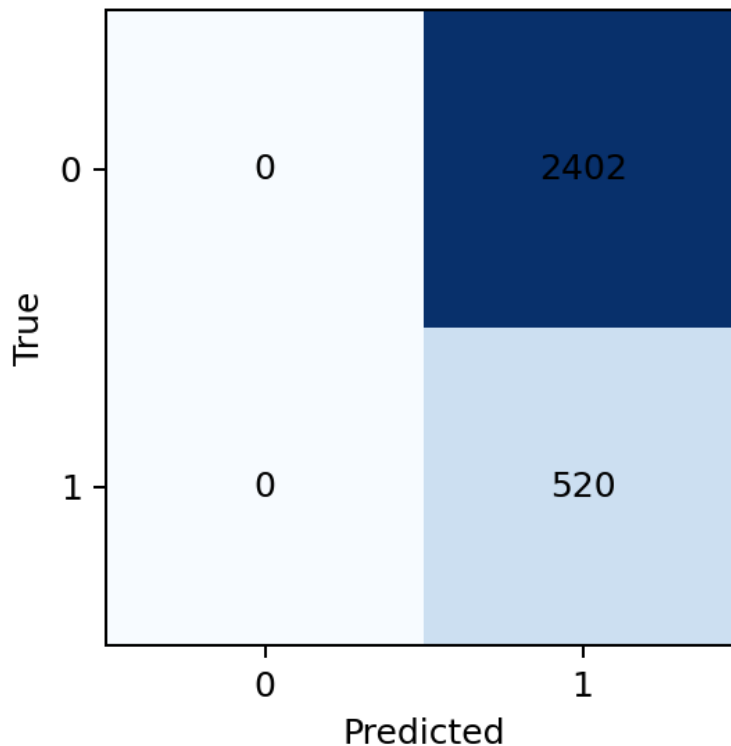


NAB EC2 CPU LSTM-AE confusion matrix: full recall with improved boundary quality.



NAB EC2 CPU CNN-AE: local motif reconstruction captures the event.

nab:ec2_cpu_utilization_5f5533 cnn_ae CM

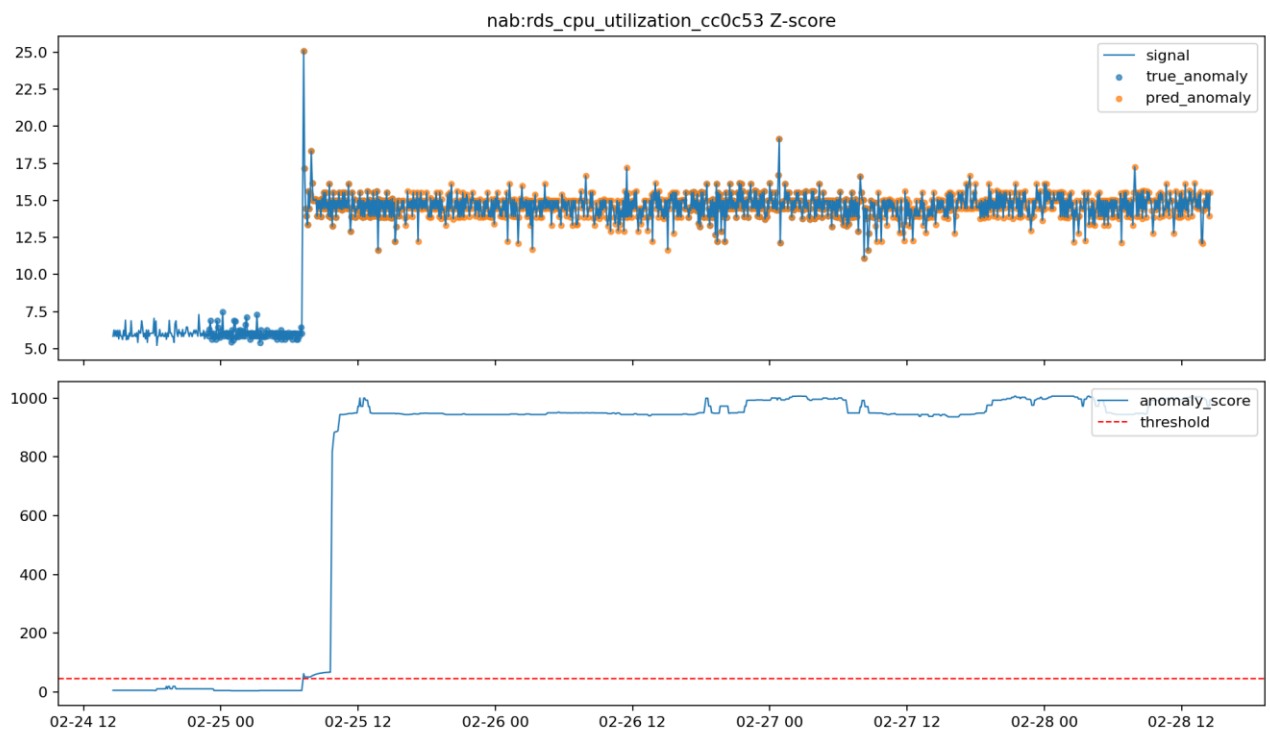


NAB EC2 CPU CNN-AE confusion matrix: similar behaviour to LSTM-AE on this split.

6.4 NAB RDS CPU Figures

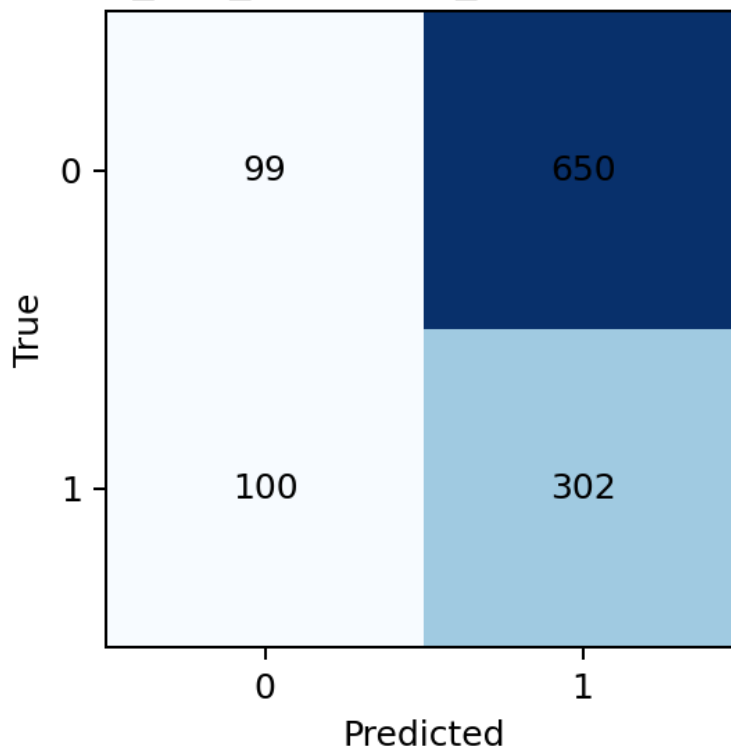
The RDS CPU series is the cleanest NAB example for the advanced models. All four models detect the event at the event level, but the autoencoders localize it more precisely point by point. Visually, this split is simpler than EC2 CPU: the anomalous interval is easier to isolate, and the main difference between models is precision rather than whether they detect the event at all.

This is the dataset where the visual evidence aligns most directly with the metric tables. The autoencoders do not just improve a secondary metric; they track the anomalous interval more tightly while keeping alerting behaviour coherent.

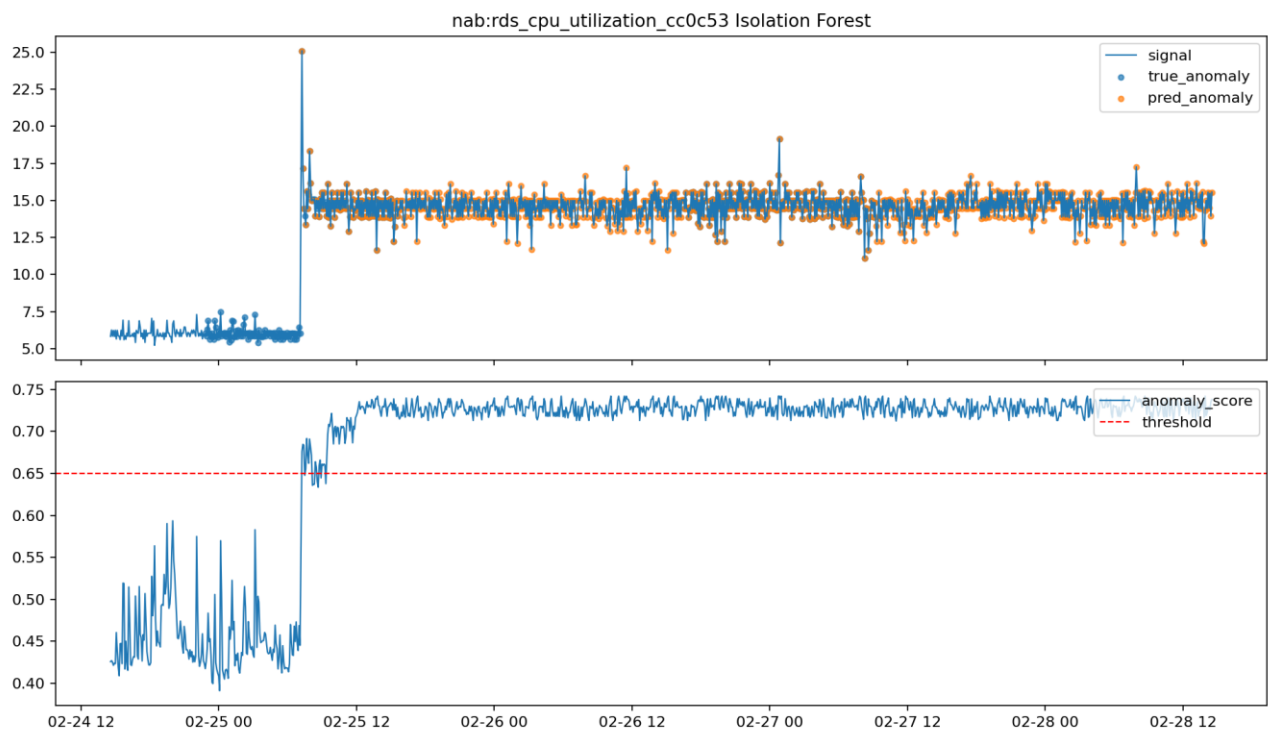


NAB RDS CPU Z-score: coherent detection with a broad alert band.

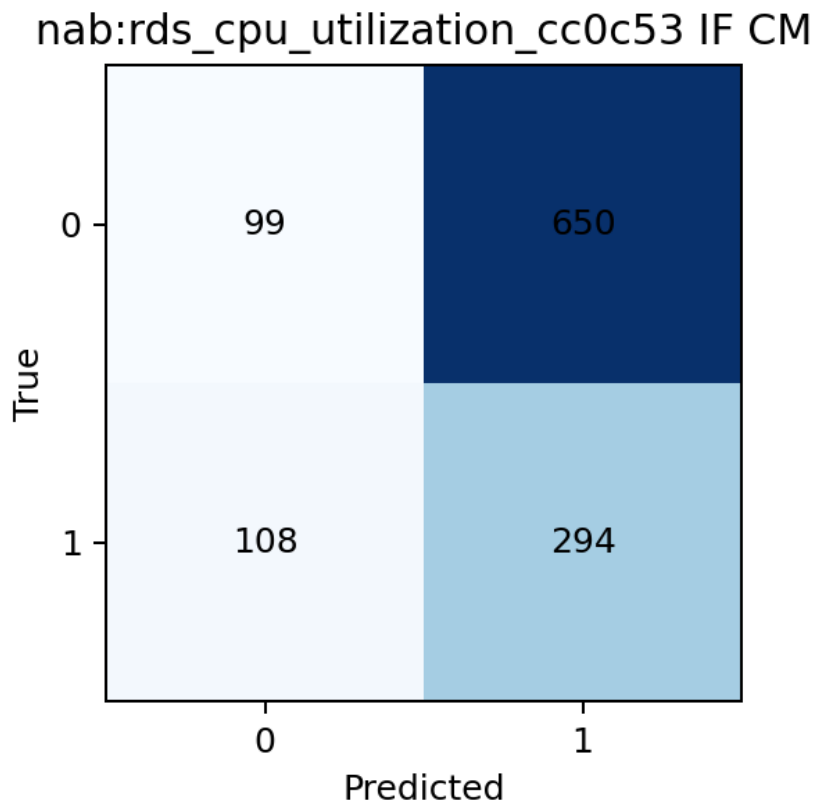
nab:rds_cpu_utilization_cc0c53 Z-score CM



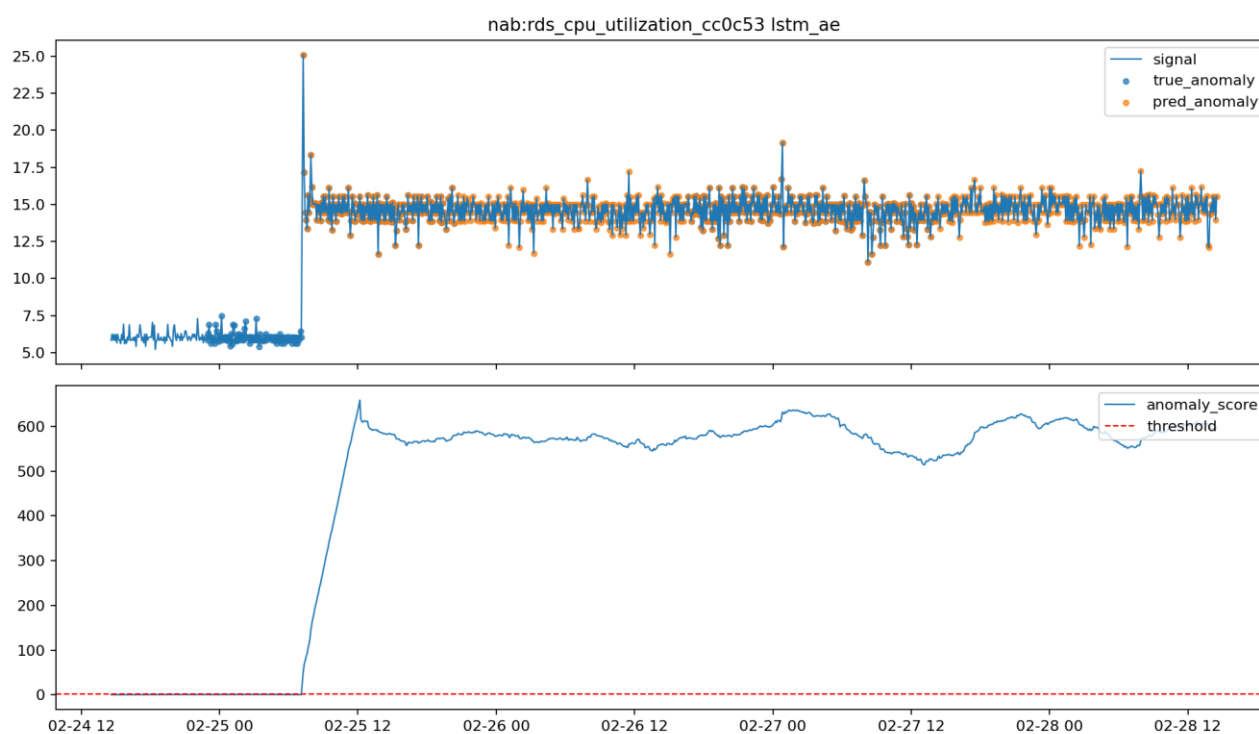
NAB RDS CPU Z-score confusion matrix: moderate precision, strong recall.



NAB RDS CPU Isolation Forest: detects the event but with some fragmentation.

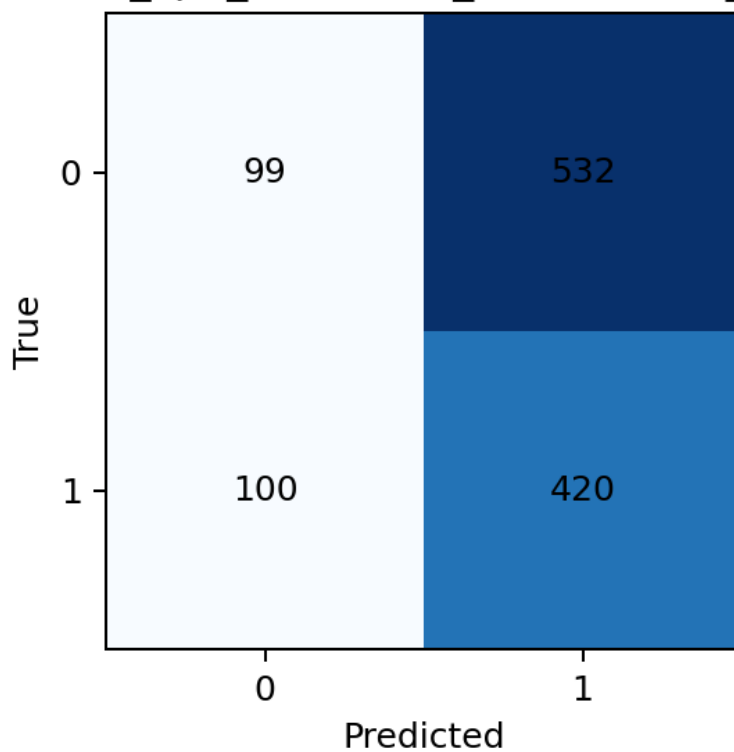


NAB RDS CPU Isolation Forest confusion matrix: similar recall with slightly noisier alerting.

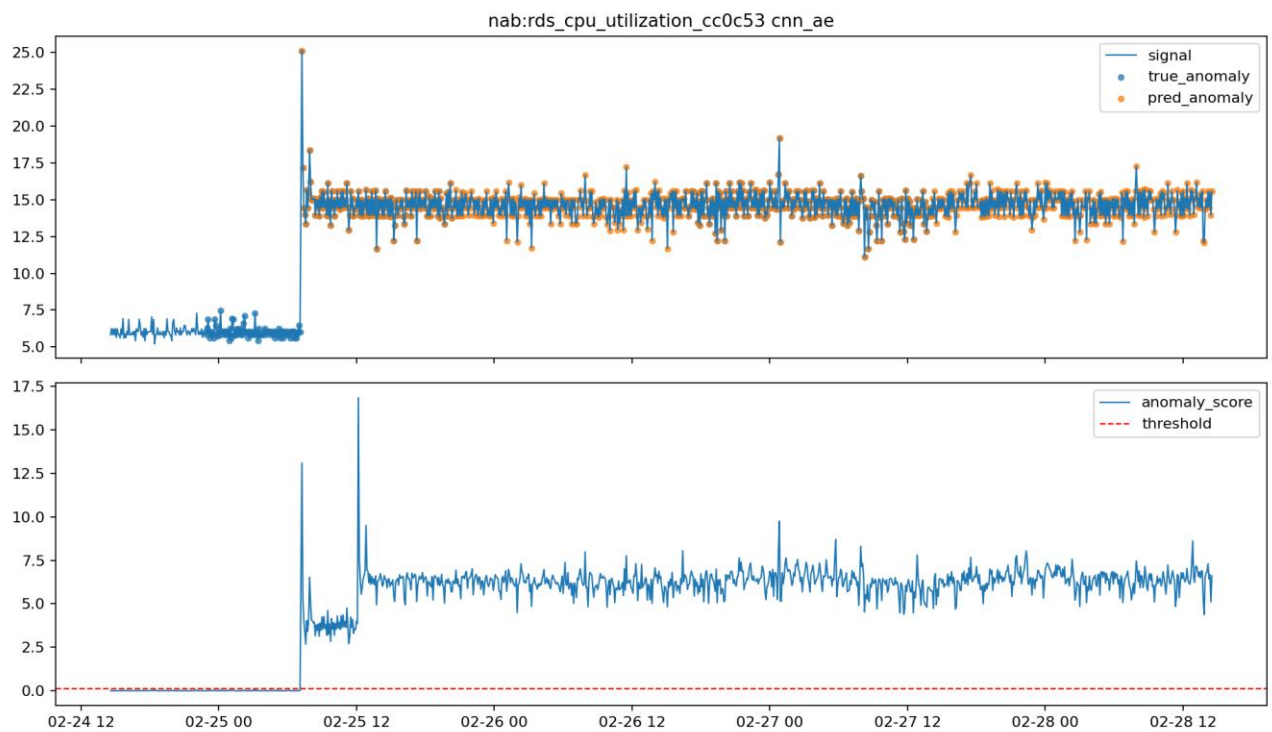


NAB RDS CPU LSTM-AE: tighter temporal localization of the event.

nab:rds_cpu_utilization_cc0c53 lstm_ae CM

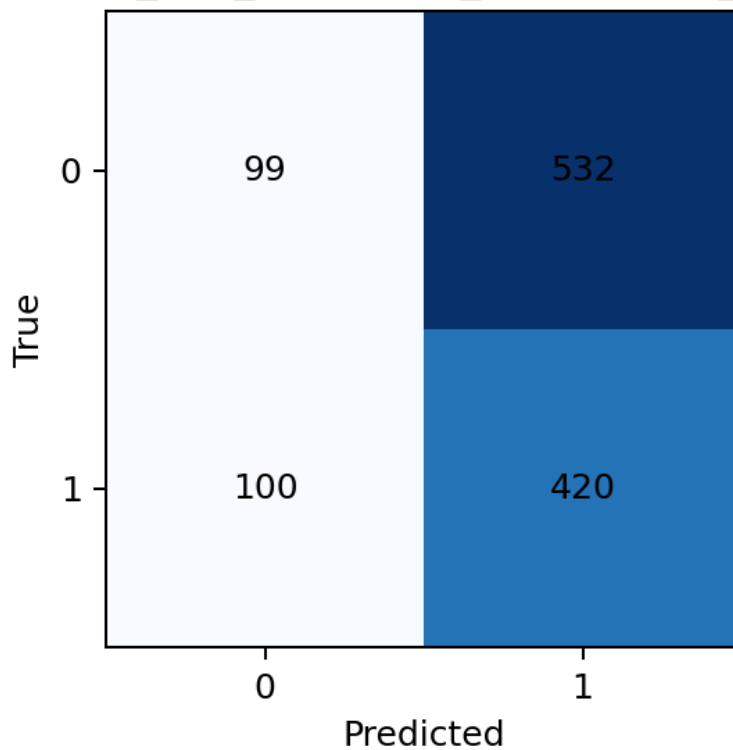


NAB RDS CPU LSTM-AE confusion matrix: strongest point-level balance on this split.



NAB RDS CPU CNN-AE: comparable performance to LSTM-AE.

nab:rds_cpu_utilization_cc0c53 cnn_ae CM

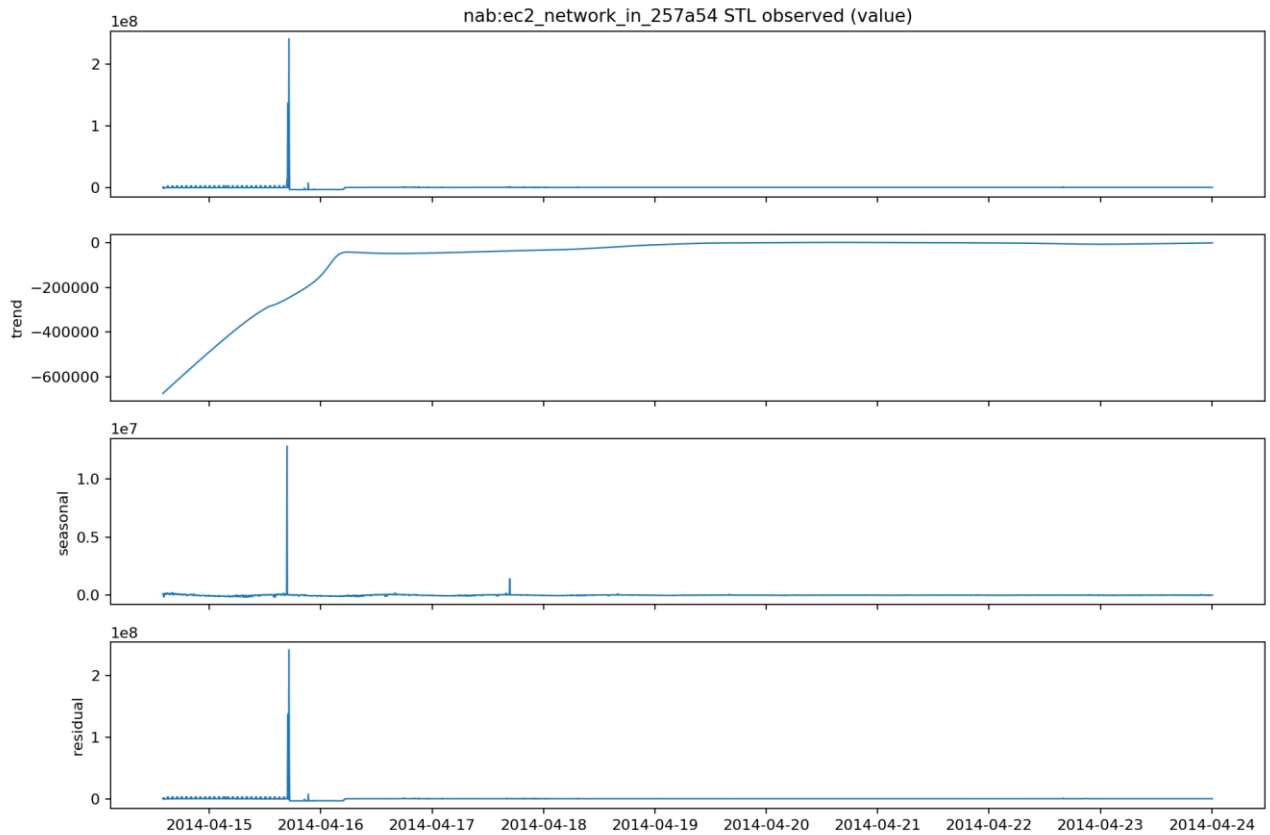


NAB RDS CPU CNN-AE confusion matrix: similarly strong precision-recall tradeoff.

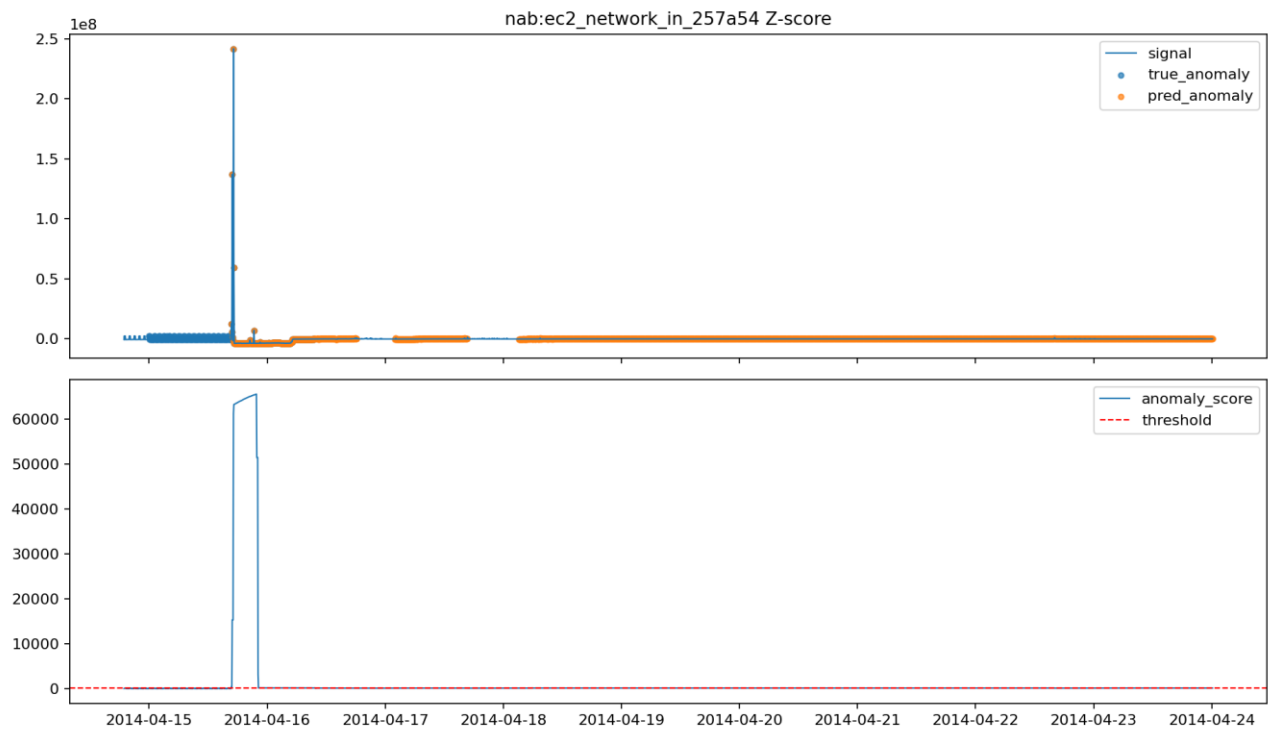
6.5 NAB EC2 Network Figures

The refreshed EC2 network run is the clearest example of why the report needed updated figures. Under the current pipeline, this split contains one collective anomaly event and now provides meaningful contrast between the models. Z-score and CNN-AE both detect the event but are either too broad or not precise enough. Isolation Forest improves point-level F1 substantially, yet the alert stream still fragments into many short events. LSTM-AE is the clearest winner because it captures the anomaly as one coherent sequence-level event while maintaining perfect precision.

This dataset strengthens the main NAB conclusion rather than acting as a special case. In noisy network telemetry, temporal modeling is especially valuable because the anomaly is easier to detect as a pattern over time than as a single extreme point.

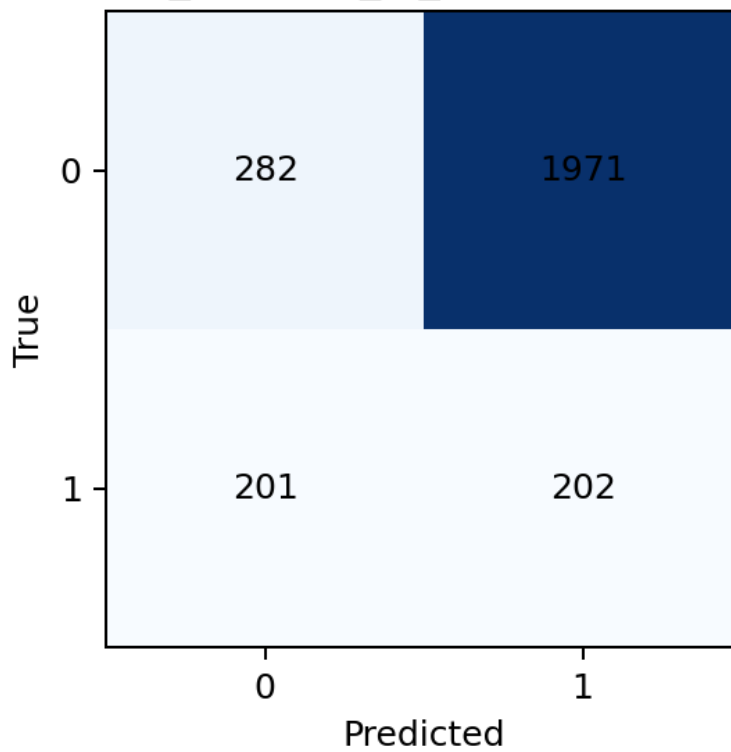


NAB EC2 Network STL decomposition: weak seasonality but clear score shift.

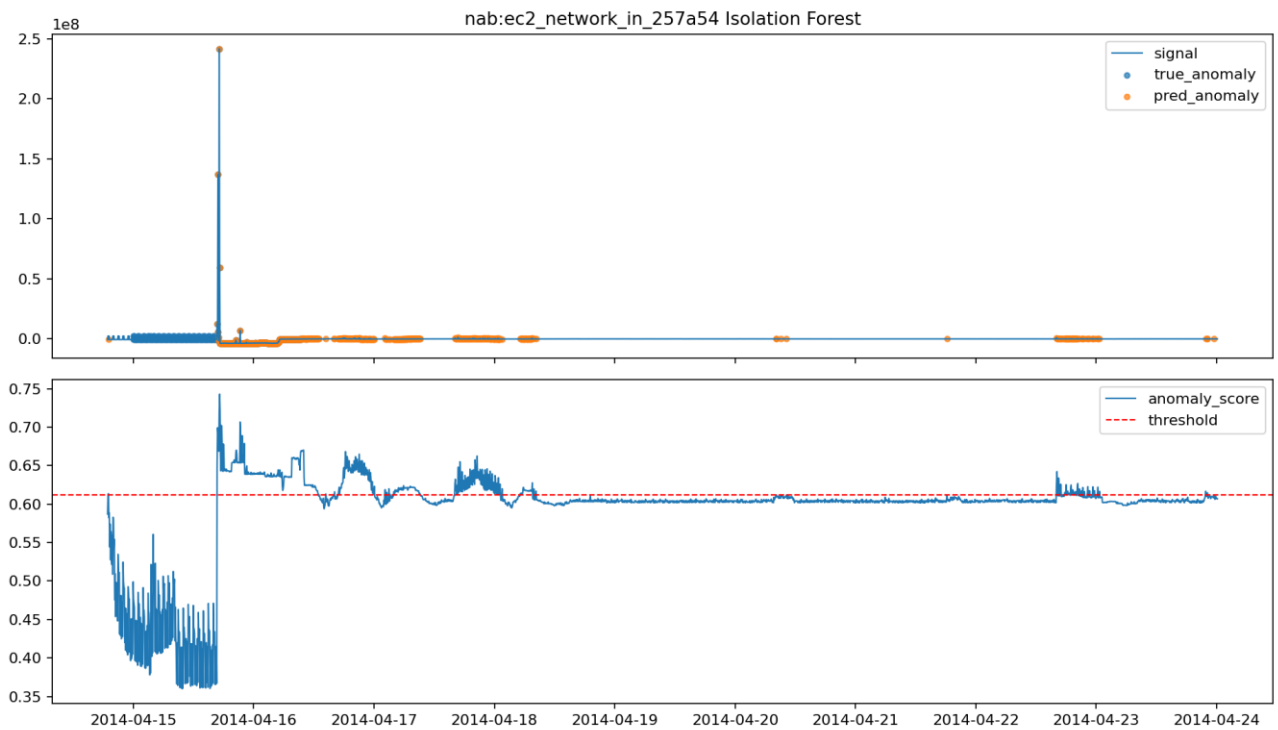


NAB EC2 Network Z-score: broad detection with several predicted segments.

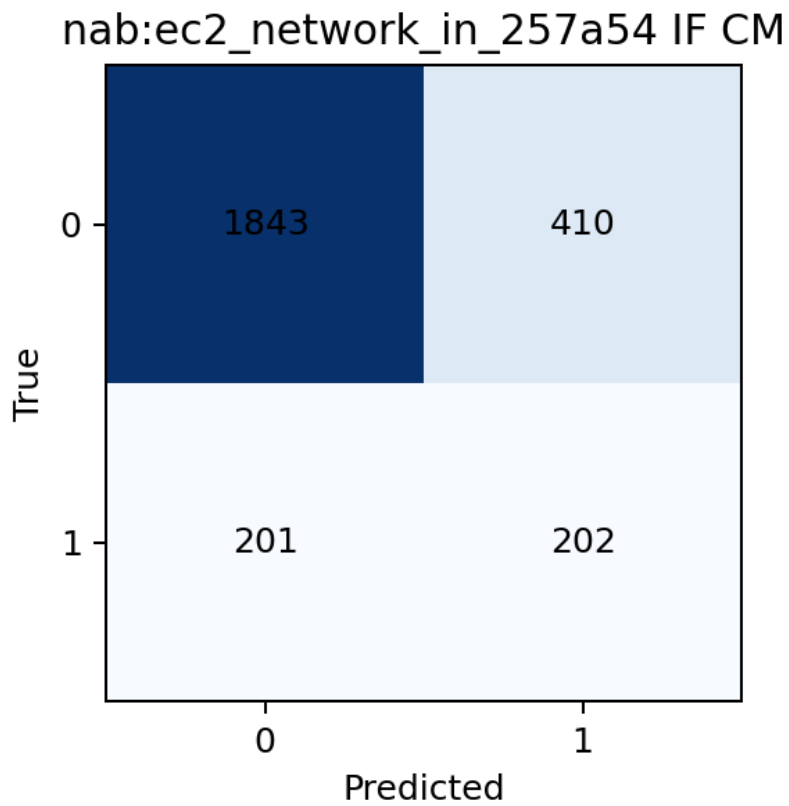
nab:ec2_network_in_257a54 Z-score CM



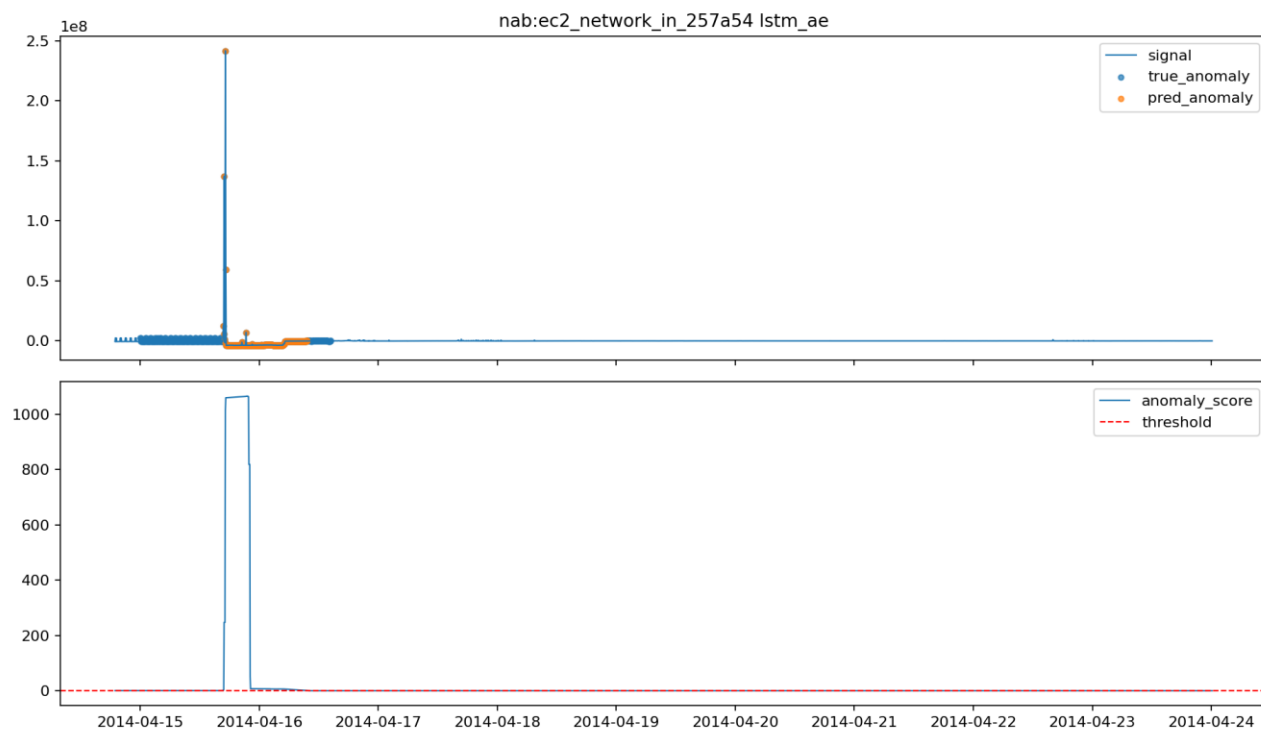
NAB EC2 Network Z-score confusion matrix: partial recall with low precision.



NAB EC2 Network Isolation Forest: stronger point detection but fragmented alerts.

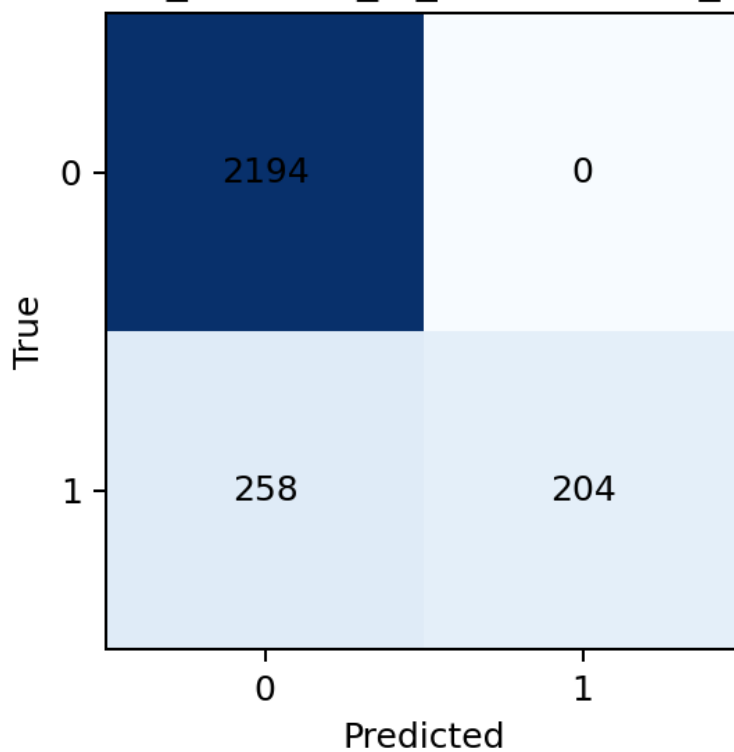


NAB EC2 Network Isolation Forest confusion matrix: improved F1 but weak event coherence.

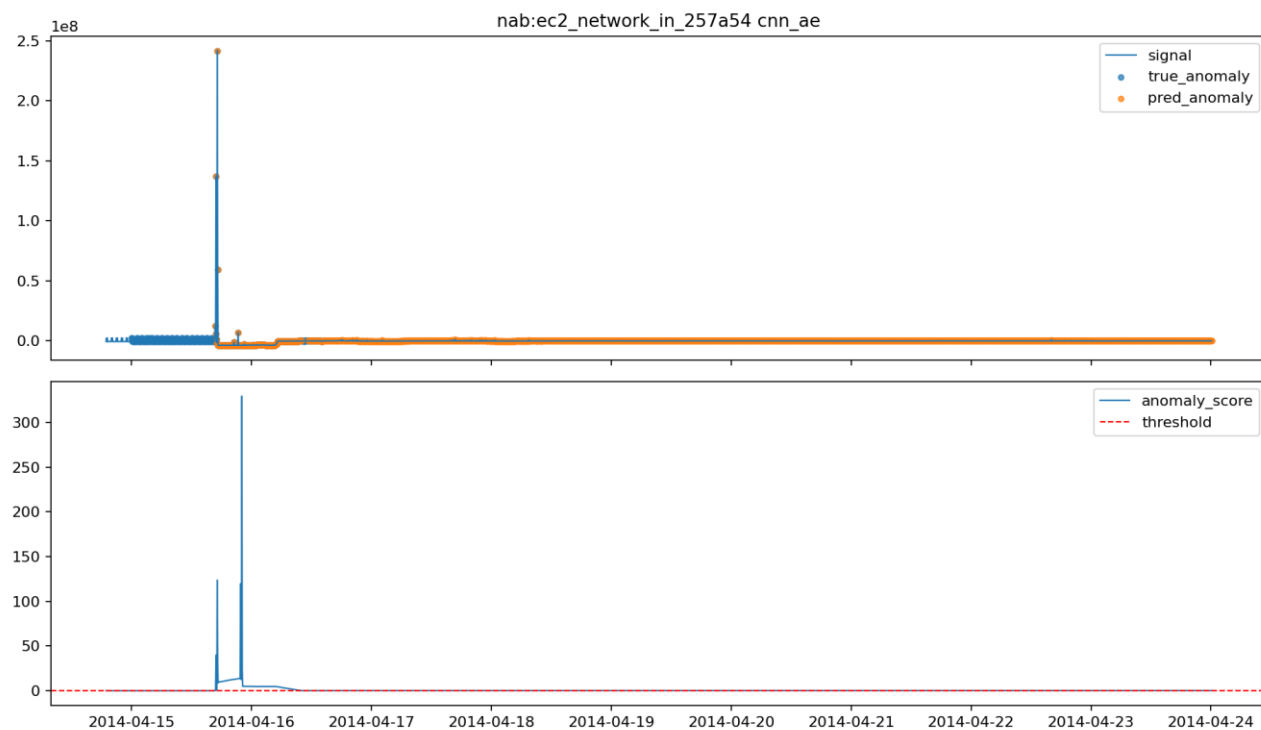


NAB EC2 Network LSTM-AE: strongest overall detection on the refreshed split.

nab:ec2_network_in_257a54 lstm_ae CM

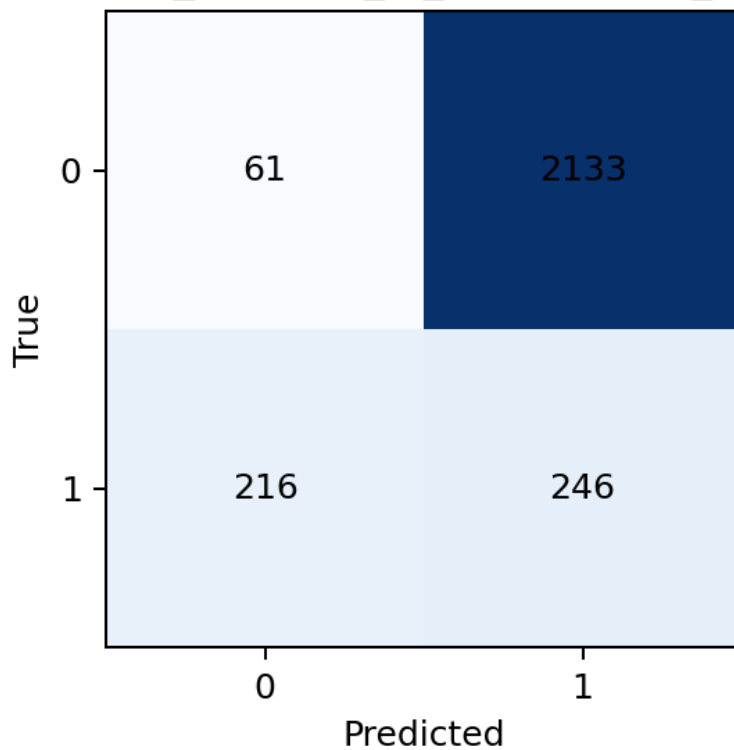


NAB EC2 Network LSTM-AE confusion matrix: perfect precision with one coherent event.



NAB EC2 Network CNN-AE: event detection with lower pointwise precision.

nab:ec2_network_in_257a54 cnn_ae CM



NAB EC2 Network CNN-AE confusion matrix: coherent event capture but broad anomaly coverage.

7. Final Discussion

7.1 Machine-Learning Findings

The main machine-learning result is that no single detector dominates in every regime. On SKAB, calibrated Z-score is the strongest operational detector because it turns one sustained fault into one coherent alert. On the noisier NAB cloud telemetry, the temporal autoencoders are usually stronger, especially when the anomaly is spread across a sequence rather than concentrated in one extreme point. Isolation Forest remains a useful baseline, but its alert stream is often more fragmented than its point-level F1 suggests.

7.2 Data and Preprocessing Findings

The report also shows that preprocessing and split design are central to performance. The anomaly-aware NAB split, rolling detrending, lag features, FFT energy, and threshold calibration all materially shape the outcome. Without those steps, the model comparison would have been much less informative. In this project, the anomaly detector is not only the final model; it is the full chain from data preparation through calibration and post-processing.

7.3 Deployment Findings

The deployment result is that the offline-to-online path works as a real system rather than only as notebook analysis. The laptop performs training, evaluation, and API hosting. The Raspberry Pi performs local inference after receiving the trained artifacts. Tailscale provides the private communication path, FastAPI provides the telemetry endpoint, and the Pi rebuilds the exact offline feature schema before scoring. This feature-schema consistency is the key technical requirement that keeps the streaming baseline path correct.

7.4 Limitations

The current system still has important limitations:

- no full Raspberry Pi latency or memory benchmark yet
- alerts are logged to CSV instead of being integrated into a monitoring stack
- thresholds are still partly tuned empirically per dataset and model
- Docker-on-Pi is available as an optional path, but it is not the central validated deployment route used in the report

7.5 Future Work

The next practical steps are:

- systematic threshold calibration rather than partly empirical tuning
- Raspberry Pi latency and memory benchmarking
- a live sensor or telemetry feed instead of replayed test-stream data
- integration with an alert dashboard or monitoring stack
- a fully Dockerized Pi deployment path if container proof becomes a formal requirement

Overall, the project demonstrates a technically viable end-to-end anomaly detection system that is interpretable, deployable, and evaluated at both the point level and the event level.

8. References

1. Lavin, A., and Ahmad, S. *Evaluating real-time anomaly detection algorithms: the Numenta Anomaly Benchmark*. IEEE ICMLA, 2015.
2. Katser, I., and Kozitsin, V. *SKAB: Skoltech Anomaly Benchmark*, 2020.
3. Liu, F. T., Ting, K. M., and Zhou, Z.-H. *Isolation Forest*. IEEE ICDM, 2008.
4. Malhotra, P., Vig, L., Shroff, G., and Agarwal, P. *LSTM-based Encoder-Decoder for Multi-sensor Anomaly Detection*. arXiv:1607.00148, 2016.